

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждения образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Программной инженерии
Специальность 6-05-0612-01 Программная инженерия (профилизация Программное обеспечение информационных технологий)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту на тему:

Web-приложение «Web-приложение для изучения иностранного языка
на основе литературных источников»

Выполнил студент Херунцева Дарья Юрьевна
(Ф.И.О.)

Руководитель проекта к.т.н., доц. В. В. Смелов
(учен. степень, звание, должность, подпись, Ф.И.О.)

Заведующий кафедрой к.т.н., доц. В. В. Смелов
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой _____

Минск 2026

Содержание

Введение	4
1 Постановка задачи и обзор аналогичных решений	5
1.1 Постановка задачи	5
1.2 Обзор аналогичных решений.....	5
1.2.1 Web-приложение «Readlang».....	6
1.2.2 Web-приложение «LingQ»	7
1.2.3 Web-приложение «Sottaku»	8
1.3 Выводы по разделу	9
2 Проектирование веб-приложения	10
2.1 Функциональные возможности	10
2.2 Логическая схема базы данных	12
2.3 Выбор платформы и технологий.....	16
2.4 Архитектура приложения.....	17
2.5 Выводы по разделу	18
3 Реализация веб-приложения	20
3.1 Программная платформа.....	20
3.2 Реляционная база данных PostgreSQL.....	20
3.3 Программные библиотеки.....	21
3.4 Структура серверной части.....	22
3.5 Реализация функций	23
3.6 Структура клиентской части.....	28
3.7 Настройка конфигурации Docker.....	28
3.8 Настройка конфигурации nginx.....	30
3.9 Выводы по разделу	31
4 Тестирование веб-приложения	32
4.1 Функциональное тестирование	32
4.2 Автоматизированное тестирование	32
4.3 Нагрузочное тестирование.....	33
4.4 Выводы по разделу	34
5 Руководство пользователя	35
5.1 Авторизация	35
5.2 Навигация по учебному контенту	36
5.3 Поиск информации	37
5.4 Личный кабинет	38
5.5 Каталог книг	39
5.6 Избранное	39
5.7 Панель администратора	40
5.8 Развертывание и проверка способности.....	46
5.9 Выводы по разделу	47
Заключение	48
Список использованной литературы	49
Приложение А Диаграмма вариантов использования	50
Приложение Б Логическая схема базы данных	51

Приложение В Архитектура приложения	52
Приложение Г Создание базы данных.....	53
Приложение Д Главный модуль.....	54

Введение

В современном обществе знание иностранных языков стало не просто преимуществом, а необходимым инструментом для профессионального и личностного роста. Многие изучающие сталкиваются с проблемой поиска качественных материалов для практики чтения – традиционные учебники часто содержат адаптированные и неестественные тексты, а чтение оригинальной литературы на начальном и среднем уровне вызывает трудности из-за обилия незнакомой лексики и грамматических конструкций. Поэтому возникает необходимость в создании специализированных цифровых платформ, способных предоставить удобный интерфейс для чтения аутентичных текстов с возможностью мгновенного перевода и запоминания новых слов.

В рамках данного проекта будет разработано веб-приложение для изучения иностранного языка на основе литературных источников, которое позволит значительно упростить процесс чтения и усвоения лексики, а также автоматизировать создание карточек для повторения. Приложение ориентировано на изучающих иностранный язык, которые хотят читать книги в оригинале, а также на преподавателей и администраторов, которые занимаются наполнением библиотеки текстами и управлением учебными материалами.

Основной целью курсового проекта является создание комплексного и интуитивно понятного веб-приложения, которое поможет повысить эффективность изучения языка через чтение, оптимизировать процессы запоминания новых слов и обеспечить комфортное погружение в аутентичную литературу.

Для достижения поставленной цели необходимо решить следующие задачи: провести постановку задачи и обзор аналогичных решений; спроектировать архитектуру приложения, обеспечивающую поддержку всех заявленных функций; разработать приложение по спроектированной архитектуре; организовать тестирование и отладку приложения для обеспечения его стабильной работы; сформировать руководство пользователя и описать инструкции по использованию веб-приложения.

Программная платформа для веб-приложения включает в себя Node.js для серверной части, а также React.js для клиентской части. Для хранения данных используется реляционная база данных SQL Server. Для контейнеризации применяется Docker Compose, а веб-сервер Nginx выполняет функции reverse-proxy и раздачи статики. Разработка осуществляется в среде Visual Studio Code.

1 Постановка задачи и обзор аналогичных решений

1.1 Постановка задачи

В современном ритме жизни люди всё чаще сталкиваются с потребностью в эффективном инструменте для изучения иностранного языка – будь то подготовка к экзамену, командировка за границу или переезд в другую страну. Этот растущий спрос на качественное языковое образование обнажает существенные пробелы в существующих системах обучения: от устаревших бумажных учебников и однотипных упражнений до неорганизованных переписок в мессенджерах и разрозненных мобильных приложений. Такие подходы не только снижают удобство для пользователей, но и лишают изучающих язык необходимой глубины погружения в аутентичный материал – особенно когда речь идёт о чтении литературы в оригинале, что напрямую влияет на прозрачность прогресса и эффективность освоения языка в целом.

Система должна поддерживать три роли пользователей: «Гость», «Пользователь» и «Администратор», каждая из которых получает доступ к определённым набору функций в соответствии с ролевой моделью.

Техническая реализация предполагает использование платформы Node.js для серверной части, React для клиентской части, SQL Server в качестве реляционной базы данных, а также Docker Compose и Nginx для развёртывания и проксирования запросов.

1.2 Обзор аналогичных решений

Для формирования целостного представления о предметной области и определения направлений разработки был выполнен обзор существующих аналогов в сфере цифрового изучения иностранных языков. На рынке представлены многочисленные веб-платформы и мобильные приложения, ориентированные на освоение лексики, грамматики и навыков чтения.

Проведенный анализ показал, что значительная часть решений либо сосредоточена на шаблонных упражнениях, либо предоставляет учебные материалы без единой структуры и сквозной траектории обучения. Во многих сервисах ограниченно реализованы персонализация, отслеживание прогресса и системная работа с аутентичными литературными источниками. Отдельной проблемой является фрагментарность функционала: поиск, учебный контент, избранное и управление данными нередко распределены между разными модулями без удобной интеграции для пользователя.

Существенным недостатком ряда аналогов также является слабая ролевой организация доступа: инструменты администрирования контента, аналитики и управления пользователями либо отсутствуют, либо реализованы неполноценно. Разрабатываемое приложение «LitLang» направлено на устранение указанных ограничений за счёт единой платформы с разграничением доступа для гостя, пользователя и администратора, структурированным представлением учебного материала, поддержкой поиска по отрывкам, ведением избранного и контролем прогресса обучения.

1.2.1 Web-приложение «Readlang»

Readlang – это веб-приложение для изучения иностранных языков через чтение литературных и любых других текстов прямо в браузере [1]. Основной принцип работы заключается в том, что пользователь кликает на незнакомое слово или фразу и мгновенно получает перевод, а система сохраняет это слово в словарь для последующего повторения. Такой подход часто приводит к фрагментарному запоминанию слов без контекста и отсутствию глубокого понимания грамматических конструкций в долгосрочной перспективе. Интерфейс веб-сайта Readlang представлен на рисунке 1.1.

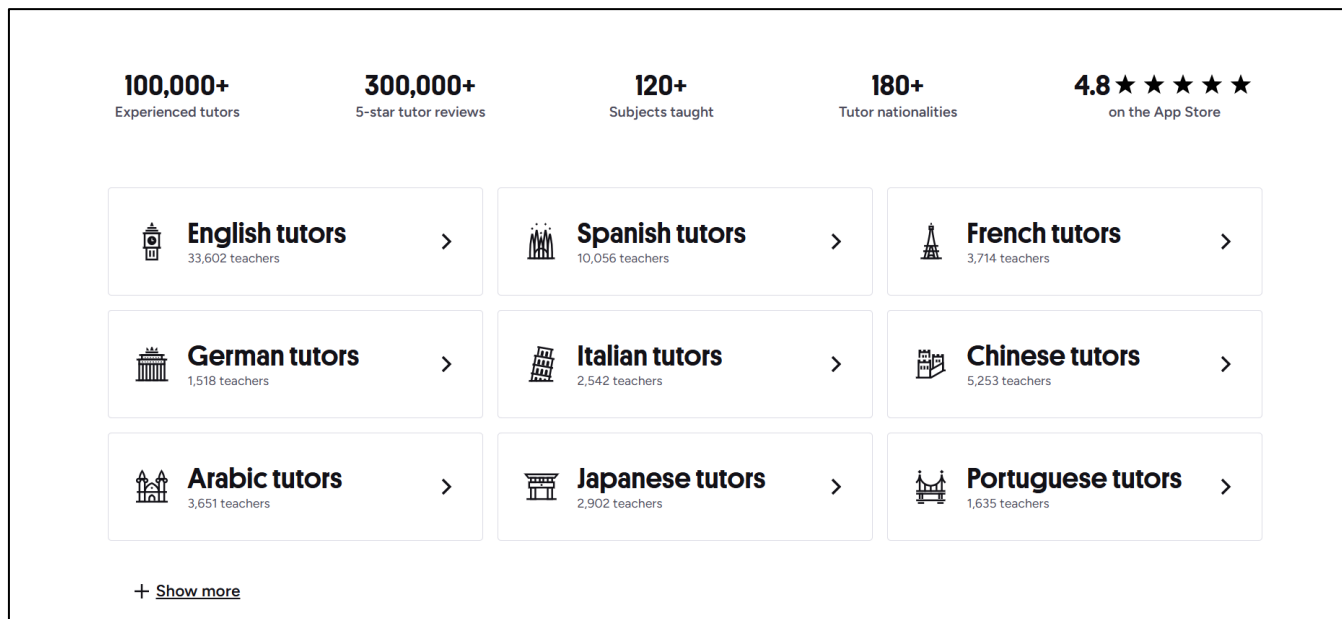


Рисунок 1.1 – Интерфейс веб-сайта «Readlang»

Достоинства:

- мгновенный перевод слова по клику; возможность импорта любых текстов, включая веб-страницы, EPUB-файлы и PDF;
- наличие встроенной системы карточек для запоминания;
- работа в любом браузере без установки дополнительных программ;
- синхронизация словаря между устройствами; поддержка десятков языков;
- контактная информация доступна в нескольких местах.

Недостатки:

- отсутствие качественной мобильной версии;
- нет встроенной библиотеки готовых;
- ограниченный функционал бесплатной версии;
- отсутствие аудиосопровождения для тренировки произношения;
- нет системы грамматических упражнений или разбора предложений;
- невозможность отслеживать прогресс по уровням сложности текстов;
- пользователь вынужден сам подбирать материалы под свой уровень.

1.2.2 Web-приложение «LingQ»

LingQ – это облачная платформа для изучения иностранных языков через чтение и прослушивание аутентичных текстов, созданная полиглотом Стивом Кауфманом [2]. Основной принцип работы заключается в том, что пользователь загружает или выбирает из библиотеки текст с аудиодорожкой, читает и слушает его одновременно, кликая на незнакомые слова, которые система подсвечивает разными цветами в зависимости от степени усвоения. Такой подход часто приводит к перегрузке интерфейса и рассеиванию внимания из-за обилия визуальных элементов и настроек. Интерфейс веб-сайта LingQ представлен на рисунке 1.2.

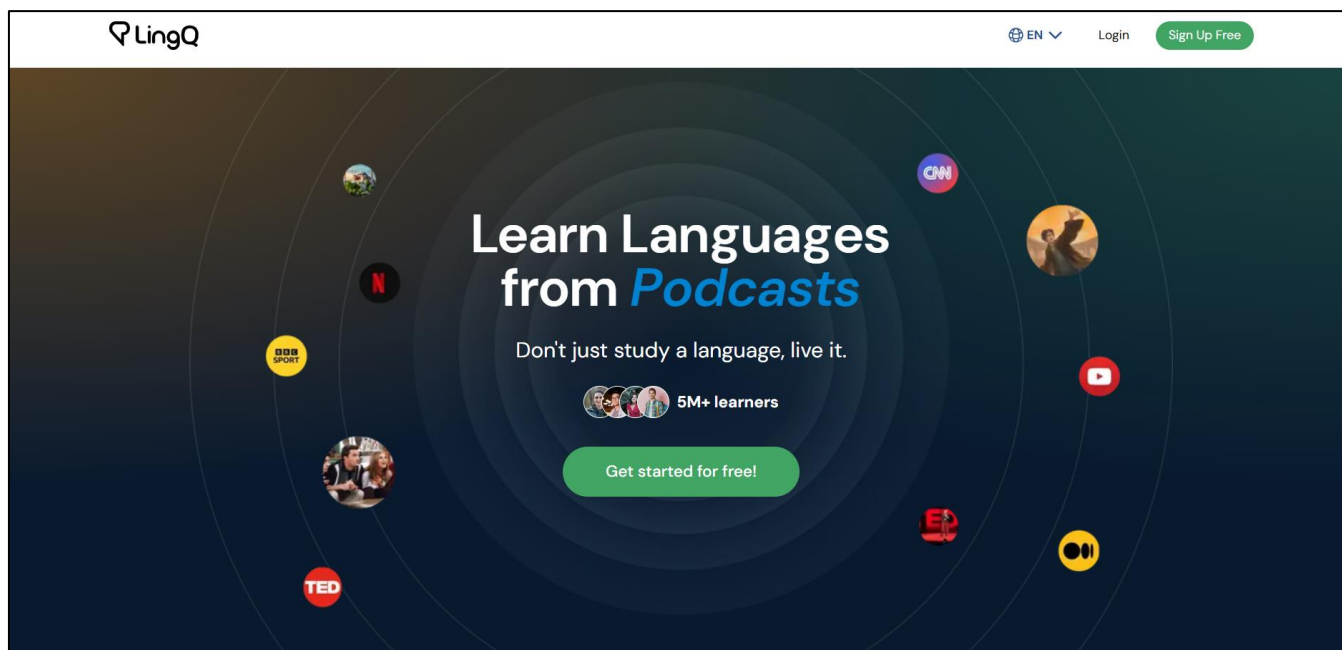


Рисунок 1.2 – Интерфейс веб-приложения «LingQ»

Достоинства:

- огромная встроенная библиотека текстов и подкастов на десятках языков;
- возможность импорта любого контента из интернета через расширение для браузера;
- одновременное чтение и прослушивание материала для развития аудирования;
- автоматическое создание карточек с контекстом для запоминания слов;
- система отслеживания статистики по изученным словам и времени занятий;
- поддержка более 40 языков, включая редкие;
- указание прайс-листа на основные и дополнительные услуги.

Недостатки:

- интерфейс перегружен и неинтуитивен для нового пользователя;
- отсутствует качественный режим чтения без лишних кнопок и панелей;
- автоматический перевод часто бывает неточным из-за опоры на машинные словари;

– нет возможности редактировать или уточнять перевод слова вручную; пользователь вынужден тратить много времени на настройку цветов и фильтров вместо самого чтения;

– отсутствует автоматическая синхронизация прогресса между веб-версией и мобильным приложением.

1.2.3 Web-приложение «Sottaku»

Sottaku – это веб-приложение и мобильная платформа для изучения иностранных языков, построенная вокруг системы интервального повторения и ориентированная прежде всего на китайский, японский, корейский и английский языки [3]. Основной принцип работы заключается в том, что пользователь изучает грамматику, проходит адаптивные тесты и читает тексты, разбитые по уровням сложности, при этом незнакомые слова автоматически сохраняются во флеш-карточки для последующего повторения по алгоритму FSRS. Такой подход часто приводит к тому, что пользователь перегружен количеством функций и настроек и тратит время не столько на чтение, сколько на освоение интерфейса. Интерфейс веб-сайта Sottaku представлен на рисунке 1.3.

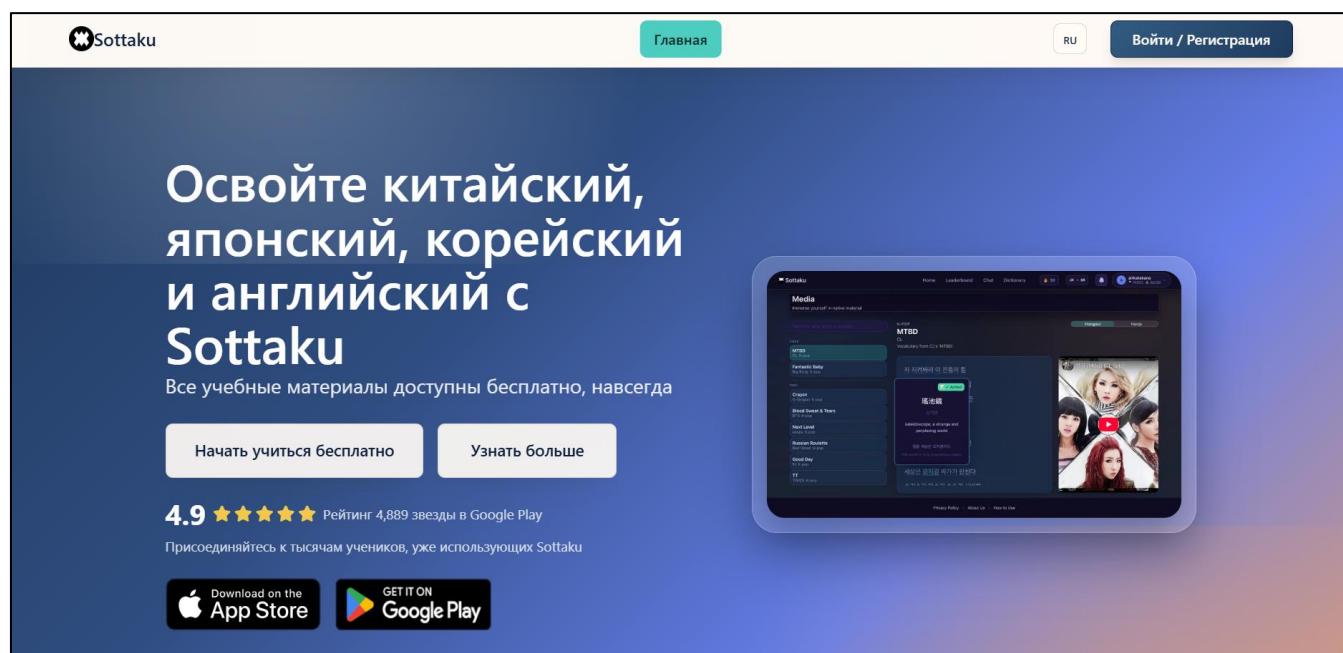


Рисунок 1.3 – Интерфейс веб-приложения «Sottaku»

Достоинства:

– полностью бесплатный доступ ко всем грамматическим руководствам, тестам и флеш-карточкам без ограничений;

– наличие расширения для браузера Sottaku-Yomitan для мгновенного перевода слов при чтении любых сайтов;

– поддержка практики письма и рукописного ввода иероглифов с отображением порядка черт;

– наличие встроенного чата для общения с другими изучающими язык;

– система адаптивных тестов и ежедневных заданий;

- синхронизация прогресса между веб-версией и мобильными приложениями;
- поддержка подготовки к экзаменам типа JLPT, HSK и Kanji Kentei.

Недостатки:

- языковой охват ограничен всего четырьмя языками – китайским, японским, корейским и английским;
- отсутствие встроенной библиотеки аудиоматериалов для тренировки аудирования;
- нет функции одновременного чтения и прослушивания текста как в LingQ ;
- пользователь вынужден самостоятельно искать и импортировать тексты;
- интерфейс может показаться перегруженным новичку из-за множества вкладок с грамматикой, тестами и карточками;
- отсутствует автоматическая проверка понимания прочитанного текста.

1.3 Выводы по разделу

1. Анализ существующих веб-приложений для изучения иностранного языка через литературные источники (Readlang, LingQ, Sottaku) показал, что они обладают рядом недостатков: отсутствие полноценной работы с художественной литературой как основным источником контента, недостаточная поддержка редких языков и отсутствие чёткого разделения ролей гостя, пользователя и администратора.

2. На основе выявленных недостатков и исходных данных листа задания разрабатываемое веб-приложение должно включать трёхуровневую ролевую модель (гость, пользователь, администратор).

3. В приложении необходимо реализовать полноценную систему изучения языка с возможностью сохранять в избранное книги, отрывки, отмечать прогресс изученных правил.

2 Проектирование веб-приложения

2.1 Функциональные возможности

Приложение поддерживает три роли пользователей: «Гость», «Пользователь», и «Администратор». Каждая роль имеет чётко определённый набор функций, соответствующий её назначению. Диаграмма вариантов использования приложения для данных ролей представлена на рисунке 2.1.

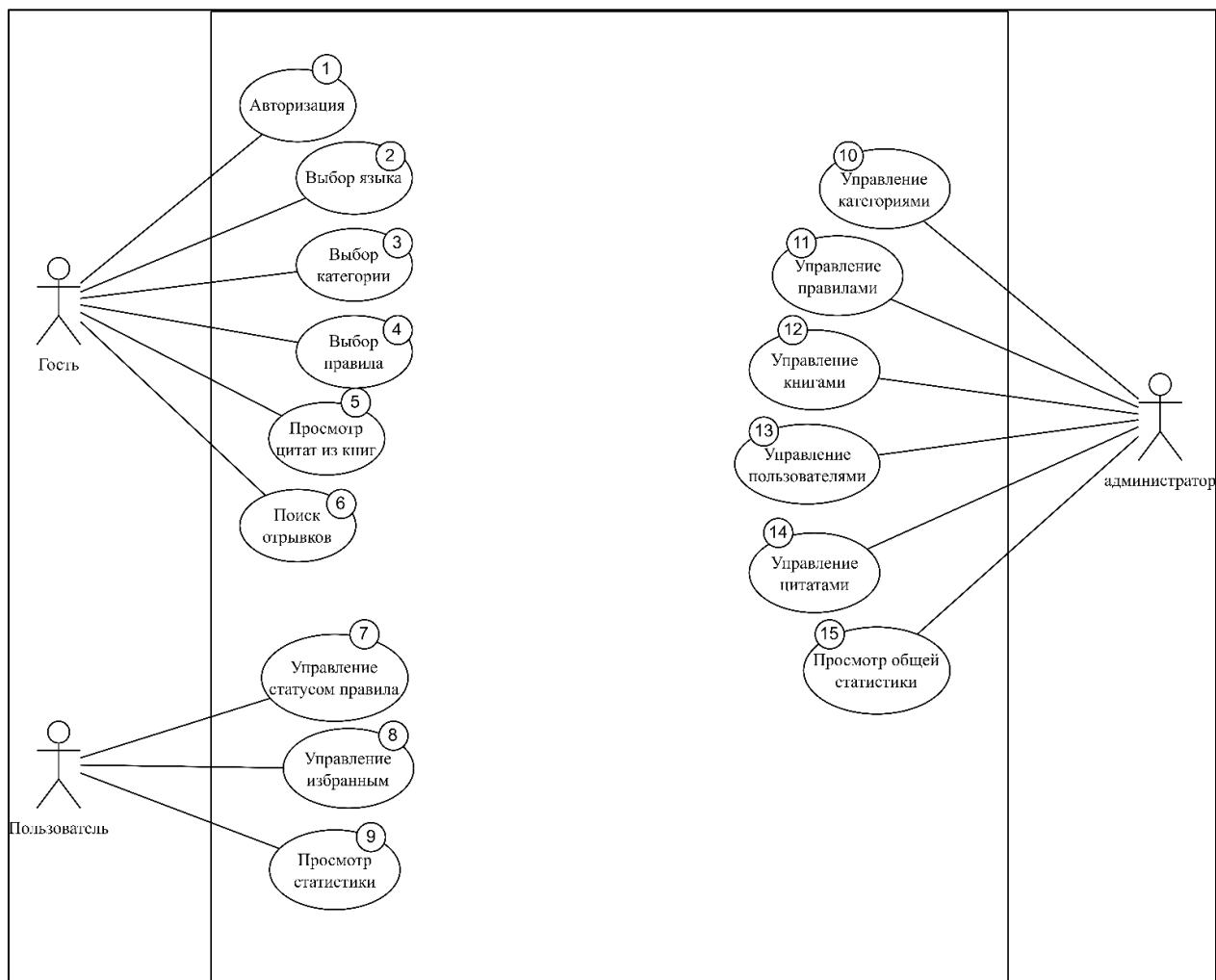


Рисунок 2.1 – Диаграмма вариантов использования

Диаграмма вариантов использования представлена в Приложении А. Описание назначения каждой роли приведено в таблице 2.1.

Таблица 2.1 – Роли пользователей приложения

Роль	Описание роли
Гость	Неавторизованный пользователь, который может просматривать отрывки, правила и категории по разным языкам.
Клиент	Авторизованный пользователь, который может добавлять отрывки и книги в избранное а также смотреть свою аналитику и отслеживать прогресс изученных правил.

Продолжение таблицы 2.1

Роль	Описание роли
Администратор	Пользователь с наибольшим количеством привилегий, который может отслеживать пользователей, статистику изучения языков, управлять книгами, правилами, отрывками

Функциональные возможности приложения для каждой роли подробно представлены в таблице 2.2.

Таблица 2.2 – Функциональные возможности приложения

№	Операция	Роль	Описание
1	авторизация	Гость	Вход в систему с использованием учетных данных
2	Выбор языка	Гость	Выбор изучаемого языка
3	Выбор категории	Гость	Выбор категории, обобщающей правила по выбранному языку
4	Выбор правила	Гость	Выбор доступного правила по выбранной категории
5	Просмотр цитат из книг	Гость	Просмотр цитат из книг по выбранному правилу с указанием названия книги, главы и страницы, а также кратким объяснением правила использованного в цитате
6	Поиск отрывков	Гость	Поиск отрывков из книг по ключевым словам
7	Управление статусом правила	Пользователь	Установление статуса правила (не изучено/в изучается/изучено)
8	Управление избранным	Пользователь	Добавление в избранное отрывков из книг, книг из библиотеки Гутенберг
9	Просмотр статистики	Пользователь	Просмотр статистики изучения языка, количества изученных/не изученных правил
10	Управление категориями	Администратор	Добавление/удаление/изменение категорий
11	Управление правилами	Администратор	Удаление/добавление/изменение правил
12	Управление книгами	Администратор	Добавление/удаление/изменение книг, загруженных из Open library или библиотеки Гутенберг
13	Управление пользователями	Администратор	Удаление пользователя, деактивация пользователя, повышение статуса пользователя до администратора
14	Управление цитатами	Администратор	Добавление/изменение/удаление цитат с возможностью создавать их с помощью искусственного интеллекта, загружать из книг в составе библиотеки Гутенберг
15	Просмотр общей статистики	Администратор	Просмотр общего количества правил, пользователей, языков и отрывков, популярных языков, степень завершенности изучения категорий, количества новых пользователей за последние 30 дней

2.2 Логическая схема базы данных

Логическая схема базы данных состоит из 12 таблиц, каждая из которых отвечает за хранение определённого типа данных. Полная логическая схема базы данных представлена в Приложении Б. Наименование и описание всех сущностей базы данных представлено в таблице 2.3.

Таблица 2.3 – Таблицы базы данных

Название таблицы	Назначение таблицы
Languages	Хранит список языков
Categories	Хранит категории правил для языков
Rules	Правила определенной категории языка
Books	Хранит список книг, добавленных администратором
Excerpts	Хранит список отрывков из книг по определенным правилам
Users	Хранит информацию о пользователях
User_progress	Хранит информацию о прогрессе пользователя (количество изученных правил, языков)
Favourites	Справочник список книг, отрывков и правил, добавленных пользователем в избранное
Rule translations	Хранит заголовки правил в переводе на различные языки
Category translations	Хранит заголовки категорий в переводе на различные языки
Gutenberg_texts	Хранит полное содержание книг, импортированных из библиотеки Гутенберг
Gutenberg_user_snippets	Хранит отрывки, из импортированных книг Гутенберг и добавленные в избранное

Далее подробно описаны основные сущности базы данных.

Описание структуры таблицы Languages, которая содержит информацию о доступных для изучения языках, приведено в таблице 2.4.

Таблица 2.4 – Структура таблицы Languages

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор языка (первичный ключ)
Code	VARCHAR(10)	Краткое название языка (EN, RU)
Name	VARCHAR(100)	Полное название языка
Created_at	NIMESTAMPTZ	Дата создания

Таблица Categories содержит информацию о категориях языка. Описание структуры данной таблицы приведено в таблице 2.5.

Таблица 2.5 – Структура таблицы Categories

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор категории (первичный ключ, автоинкремент)
Language_id	INTEGER	Уникальный идентификатор языка, к которому относится категория
Title	VARCHAR(200)	Название категории, отображаемое на странице
Slug	VARCHAR(200)	Человекочитаемый технический идентификатор написанный латиницей через дефисы

Продолжение таблицы 2.5

Поле	Тип	Назначение
Sort_order	INTEGER	Порядковый номер отображения на странице относительно других категорий
Created_at	TIMESTAMPTZ	Дата создания

Таблица Rules хранит правила которые относятся к какой-либо категории. Описание структуры данной таблицы приведено в таблице 2.6.

Таблица 2.6 – Структура таблицы Rules

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор правила
Slug	VARCHAR(300)	Человекочитаемый технический идентификатор написанный латиницей через дефисы
Category_ID	INTEGER	Описание типа животного
Title	VARCHAR(300)	Уникальный идентификатор категории правила
Summary	TEXT	Краткое описание правила
Sort_order	INTEGER	Номер отображения правила на странице
Created_at	TIMESTAMPTZ	Дата создания

Таблица Books содержит информацию о добавленных администратором книгах. Описание структуры таблицы приведено в таблице 2.7.

Таблица 2.7 – Структура таблицы Books

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор книги (первичный ключ, автоинкремент)
Title	VARCHAR(500)	Название книги
Author	VARCHAR(300)	Автор книги
ISBN	VARCHAR(20)	Уникальный идентификатор isbn книги
Language_id	INTEGER	Уникальный идентификатор языка
Cover_url	VARCHAR(500)	Ссылка на обложку книги
CreatedAt	TIMESTAMPTZ	Дата и время создания записи

Таблица Excerpts хранит информацию об отрывках. Описание структуры данной таблицы приведено в таблице 2.8.

Таблица 2.8 – Структура таблицы Excespts

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор отрывка (первичный ключ, автоинкремент)
Book_Id	INTEGER	Уникальный идентификатор книги
Rule_ID	INTEGER	Уникальный идентификатор правила
Highlight	TEXT	Выделенная часть отрывка
Page Number	TEXT	Номер страницы
Chapter	VARCHAR(10)	Глава
Context note	VARCHAR(100)	Описание правила
Created_at	TIMESTAMPTZ	Дата добавления
Sort_order	INTEGER	Порядковый номер отображения на странице

Таблица Users содержит информацию о пользователях. Описание структуры данной таблицы приведено в таблице 2.9.

Таблица 2.9 – Структура таблицы Users

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор пользователя (первичный ключ)
Email	VARCHAR(225)	Почта, указанная пользователем при регистрации
Password hash	VARCHAR(225)	Пароль, хранимый в хэше
Display name	VARCHAR(100)	Отображаемое имя
Role	VARCHAR(20)	Роль(пользователь/администратор/гость)
Is active	BOOLEAN	Является ли аккаунт активным
Created at	TIMESTAMPTZ	Дата создания аккаунта
Updated at	TIMESTAMPTZ	Дата последнего обновления аккаунта

Таблица User_progress хранит информацию о количестве изученных и изучаемых в данный момент правил. Описание структуры данной таблицы приведено в таблице 2.10.

Таблица 2.10 – Структура таблицы User_progress

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор прогресса(первичный ключ)
User_id	UUID	Уникальный идентификатор пользователя
Rule_id	INTEGER	Уникальный идентификатор правила
Status	VARCHAR(20)	Статус правила(изучено/не изучено/изучается)
Updated_at	TIMESTAMPTZ	Дата последнего обновления прогресса

Таблица Favourites является справочником возможных статусов бронирования. Описание структуры данной таблицы приведено в таблице 2.11.

Таблица 2.11 – Структура таблицы Favourites

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор объекта (первичный ключ)
User_id	UUID	Уникальный идентификатор пользователя
Target type	VARCHAR(20)	Тип объекта
Target id	INTEGER	ID целевой сущности в соответствующей таблице
Created_at	TIMESTAMPTZ	Дата добавления объекта в избранное

Таблица Gutenberg_texts хранит полное содержание импортированных книг из библиотеки Гутенберг. Описание структуры данной таблицы приведено в таблице 2.12.

Таблица 2.12 – Структура таблицы Gutenberg_texts

Поле	Тип	Назначение
Id	SERIAL(PK)	Идентификатор строки с полным текстом
Book_id	INTEGER	К какой книге относится скачанный текст
Full_text	TEXT	Полный очищенный текст Gutenberg для поиска/вырезки отрывков
Created_at	TIMESTAMPTZ	Дата импорта книги из библиотеки Гутенберг

Таблица Gutenberg_user_snippets хранит отрывки, ранее запрошенные пользователем из импортированной из библиотеки Гутенберг книги и позже добавленного в избранное. Описание структуры данной таблицы приведено в таблице 2.13.

Таблица 2.13 – Структура таблицы Gutenberg_user_snippets

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор отрывка (первичный ключ)
User_id	UUID	Уникальный идентификатор пользователя
Book_id	INTEGER	Уникальный идентификатор книги
Passage	TEXT	Текст отрывка
paragraph_number	INTEGER	Условный номер абзаца/позиции в разбиении
highlight	TEXT	Подсвечиваемый отрывок
Page_number	VARCHAR(50)	Номер страницы
Chapter	VARCHAR(200)	Номер главы
Rule_id	INTEGER	Уникальный идентификатор правила
created_at	TIMESTAMPTZ	Дата добавления отрывка в избранное

Таблица Category_translations хранит заголовки категории в указанном языке. Описание структуры данной таблицы приведено в таблице 2.14.

Таблица 2.14 – Структура таблицы Category_translations

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор записи (первичный ключ)
Category_id	INTEGER	Уникальный идентификатор категории
Locale	VARCHAR(10)	Локаль (en, ru, ...)
Title	VARCHAR(200)	Заголовок категории на данном языке

Таблица Rule_translations заголовок правила в указанном языке. Описание структуры данной таблицы приведено в таблице 2.15.

Таблица 2.15 – Структура таблицы Rule_translations

Поле	Тип	Назначение
Id	SERIAL(PK)	Уникальный идентификатор записи (первичный ключ)
Rule_id	INTEGER	Уникальный идентификатор правила
Locale	VARCHAR(10)	Локаль (en, ru, ...)
Title	VARCHAR(300)	Заголовок правила на данном языке
Summary	TEXT	Текст объяснения/описания правила

Связи между таблицами базы данных представлены в таблице 2.16.

Таблица 2.16 – Связи между таблицами базы данных

Таблица источник	Связанная таблица	Тип связи
Languages	Categories	Один ко многим
Languages	Books	Один ко многим
Categories	Category translations	Один ко многим
Categories	Rules	Один ко многим

Продолжение таблицы 2.16

Таблица источник	Связанная таблица	Тип связи
Rules	Rule translations	Один ко многим
Rules	Excerpts	Один ко многим
Books	Excerpts	Один ко многим
Users	User progress	Один ко многим
Rules	User progress	Один ко многим
Users	Favourites	Один ко многим
Books	Gutenberg texts	Один ко многим
Books	Gutenberg user snippets	Один ко многим
Users	Gutenberg user snippets	Один ко многим
Rules	Gutenberg user snippets	Один ко многим
Languages	Categories	Один ко многим
Languages	Books	Один ко многим
Categories	Category translations	Один ко многим

Разработанная логическая схема базы данных включает 12 таблиц, полностью покрывающих все функциональные требования к системе.

2.3 Выбор платформы и технологий

Веб-приложение «LitLang» предназначено для работы в современных браузерах, что обеспечивает кроссплатформенность и доступность на различных устройствах.

Бэкенд (серверная часть):

- Node.js (версия 20 LTS) – платформа для выполнения JavaScript на сервере, обеспечивающая асинхронную обработку запросов.

- Express (версия 4.21.0) – веб-фреймворк для Node.js, упрощающий создание REST API.

- Knex (версия 3.1.0) – построитель SQL-запросов и инструмент миграций для работы с реляционной базой данных.

- bcrypt (версия 5.1.1) – библиотека для хеширования паролей.

- jsonwebtoken (версия 9.0.2) – реализация JWT для аутентификации.

- express-validator (версия 7.2.0) – валидация входных данных запросов к API.

- pg (версия 8.13.0) – драйвер для подключения к PostgreSQL.

Фронтенд (клиентская часть):

- React (версия 19.2.4) – библиотека для построения пользовательских интерфейсов.

- React Router DOM (версия 7.13.1) – маршрутизация в клиентском приложении.

- Axios (версия 1.13.6) – HTTP-клиент для запросов к серверу.

- Vite (версия 8.0.0) – сборка и dev-сервер клиентского приложения.

- Tailwind CSS (версия 3.4.19) – утилитарная стилизация интерфейса.

- i18next и react-i18next (версии 25.8.18 и 16.5.8) – многоязычный интерфейс приложения.

База данных:

– PostgreSQL 16 (образ Alpine в Docker Compose) – реляционная СУБД, обеспечивающая ACID-транзакции и надёжное хранение данных.

Инфраструктура:

– Docker и Docker Compose – контейнеризация приложения (сервисы API, база данных, Nginx).

– Nginx (официальный образ nginx:alpine) – веб-сервер для раздачи статики фронтенда и reverse-proxy к API.

– JWT (JSON Web Tokens) – механизм аутентификации.

Таким образом, выбранный стек технологий обеспечивает масштабируемость, безопасность данных и удобство развёртывания приложения с помощью контейнеризации.

2.4 Архитектура приложения

Архитектура веб-приложения построена по клиент-серверному принципу с использованием REST API. Архитектурная схема представлена на рисунке 2.2 и в Приложении В.

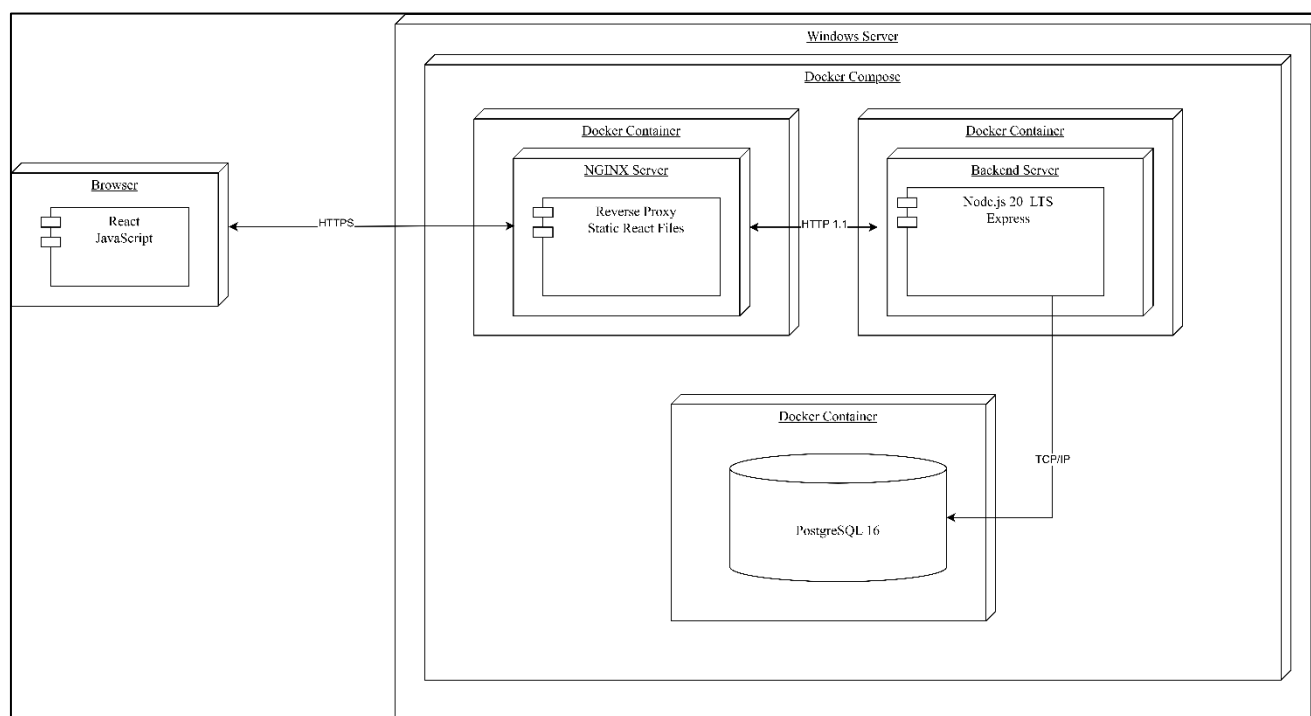


Рисунок 2.2 – Архитектура приложения

В таблице 2.17 представлены ключевые компоненты архитектуры разработанного веб-приложения, их технологическая основа и функциональная роль в системе.

Таблица 2.17 – Основные компоненты архитектуры

Компонент	Технология	Назначение
Клиент	React (Vite), React Router DOM, Axios, Tailwind CSS	Формирование пользовательского интерфейса, маршрутизация между страницами, отправка HTTP-запросов к API, отображение результатов
Веб-сервер	Nginx	Раздача статических файлов клиентского приложения, reverse-проxy для API-запросов, точка входа для внешних подключений
Сервер приложений	Node.js + Express.js	Реализация бизнес-логики, обработка REST-запросов, валидация данных, аутентификация пользователей (JWT), интеграция с внешними сервисами (Gutenberg, Open Library, AI)
База данных	PostgreSQL,	Надежное хранение данных приложения (пользователи, книги, правила, отрывки, избранное, прогресс), выполнение транзакций и запросов

Описание сетевого взаимодействия между слоями системы представлено в таблице 2.18.

Таблица 2.18 – Протоколы взаимодействия

Протокол	Назначение
HTTP/HTTPS	Обмен данными между клиентом и Nginx; проксирование запросов от Nginx к серверу приложений
TCP (порт 5432)	Взаимодействие сервера приложений с PostgreSQL

Выбранная архитектура обеспечивает четкое разделение ответственности между уровнями: клиент отвечает за представление данных, веб-сервер — за доставку фронтенда и маршрутизацию трафика, сервер приложений — за предметную логику, а база данных — за целостное и устойчивое хранение информации. Такой подход повышает сопровождаемость решения и упрощает масштабирование: каждый слой можно развивать независимо без полной переработки всей системы.

2.5 Выводы по разделу

1. Веб-приложение «Litlang» реализует ролевую модель доступа с тремя основными типами пользователей: гость, авторизованный пользователь и администратор.

2. Функциональность системы охватывает основные пользовательские сценарии: работу с правилами и отрывками, поиск и импорт книг, сохранение материала в избранное, отслеживание прогресса обучения.

3. Логическая модель базы данных включает основные сущности предметной области: пользователи, языки, категории, правила, переводы, книги, отрывки, избранное, прогресс, тексты Gutenberg и пользовательские отрывки, что обеспечивает покрытие требований учебного сервиса. Определены структуры

всех таблиц с указанием типов полей, а также связи между таблицами (один ко многим, один к одному).

4. В качестве технологического стека выбраны: Node.js + Express (серверная часть), React + Vite (клиентская часть), PostgreSQL (СУБД), Nginx (reverse proxy), Docker Compose (контейнеризация). Для доступа к БД используется Knex.js.

5. Архитектура построена по клиент-серверному принципу с использованием REST API, что обеспечивает масштабируемость, расширяемость и удобство сопровождения системы.

Такое архитектурное решение позволяет разделить интерфейсный и серверный уровни, упростить развитие функциональности и независимое обновление компонентов. Контейнеризация гарантирует воспроизводимость окружения при развертывании, а Nginx выступает единой точкой входа для клиентской части и API. Безопасность учетных записей обеспечивается за счет JWT-аутентификации и хеширования паролей с использованием bcrypt.

3 Реализация веб-приложения

3.1 Программная платформа

Для серверной части проекта выбрана платформа Node.js (20 LTS) [4], ориентированная на асинхронную обработку запросов и эффективную работу с сетевыми операциями. В качестве веб-фреймворка используется Express.js (4.21.0) [5], обеспечивающий построение REST API [6], настройку маршрутов и подключение промежуточных обработчиков (middleware).

Клиентская часть реализована на библиотеке React (19.2.4) [7] с компонентным подходом к построению интерфейса. Для разработки и сборки используется Vite [8], обеспечивающий быстрый запуск в режиме разработки и оптимизированную production-сборку. Для взаимодействия с сервером применяются HTTP-запросы через Axios, а маршрутизация в интерфейсе организована с помощью React Router [9].

3.2 Реляционная база данных PostgreSQL

В качестве системы управления базами данных в проекте используется PostgreSQL (16) [10], обеспечивающая надежное хранение структурированных данных и поддержку ACID-транзакций. В базе данных хранятся сущности предметной области: пользователи, языки, категории, правила, книги, отрывки, избранное, прогресс обучения и тексты Gutenberg [11].

Подключение к PostgreSQL в серверной части реализовано через Knex.js. Данный инструмент обеспечивает:

- безопасную параметризацию запросов;
- удобное построение SQL-выборок;
- поддержку миграций и переносимой конфигурации между окружениями.

Конфигурация подключения к базе данных представлена в листинге 3.1.

```
const knex = require('knex');
const knexConfig = require('../knexfile');

const env = process.env.NODE_ENV || 'development';
const db = knex(knexConfig[env]);

module.exports = db;
```

Листинг 3.1 – Подключение к PostgreSQL через Knex

В листинге 3.1 показано создание единого экземпляра подключения к базе данных на основе окружения (development/production).

Это позволяет централизованно использовать подключение во всех сервисах приложения.

Пример безопасного параметризованного запроса, который осуществляет поиск отрывков из книг, учитывая строковый запрос, введенный пользователем приведен в листинге 3.2.

```

async function searchExcerpts(searchQuery, pagination) {
  const q = searchQuery.trim();
  const likePattern = `%${q.replace(/\\/g, '\\\\').replace(/%/g,
'\\%').replace(/_/g, '\\_')}%`;
  const query = db('excerpts as e')
    .join('books as b', 'b.id', 'e.book_id')
    .where((wb) => {
      wb.whereRaw(
        "to_tsvector('english',
e.passage)
plainto_tsquery('english', ?)",
[q]
      )
      .orWhere('e.passage', 'ilike', likePattern)
      .orWhere('b.title', 'ilike', likePattern)
      .orWhere('b.author', 'ilike', likePattern);
    })
    .select(
      'e.id',
      'e.passage',
      'e.highlight',
      'e.rule_id',
      'b.title as book_title',
      'b.author as book_author'
    )
    .orderBy('e.id', 'desc');
  return paginate(query, pagination);
}

```

Листинг 3.2 – Пример параметризованного запроса в сервисе

В листинге 3.2 показано, что входные параметры передаются в запрос в параметризованном виде, а строка поиска предварительно экранируется. Это позволяет безопасно выполнять полнотекстовый и частичный поиск по тексту отрывка, названию книги и имени автора без риска SQL-инъекций. Запрос возвращает массив найденных записей с основной информацией об отрывке и книге, после чего результат обрабатывается пагинацией.

Структура базы данных в проекте формируется и поддерживается с помощью миграций Knex[12]; полный набор миграций и seed-данных представлен в каталоге server/src/db/migrations и server/src/db/seeds.

3.3 Программные библиотеки

Для реализации функциональности приложения используются серверные и клиентские библиотеки.

Их выбор обусловлен требованиями к безопасности, удобству разработки, расширяемости и производительности.

В таблице 3.1 приведены ключевые библиотеки серверной части.

Таблица 3.1 – Список библиотек (серверная часть)

Библиотека	Версия	Назначение
express	4.21.0	Веб-фреймворк для реализации REST API
knex	3.1.0	Построение SQL-запросов и работа с миграциями
pg	8.13.0	Драйвер подключения к PostgreSQL
bcrypt	5.1.1	Хеширование паролей пользователей
jsonwebtoken	16.4.7	Создание и проверка JWT-токенов
cors	2.8.5	Настройка междоменных запросов
express-validator	7.2.1	Валидация входящих данных
helmet	7.1.0	Базовая защита HTTP-заголовков
express-rate-limit	7.4.0	Ограничение частоты запросов
axios	1.7.0	HTTP-клиент для интеграций (Gutenberg, Open Library, AI)
pino / pino-http	9.4.0 / 10.3.0	Логирование и HTTP-логирование

Как следует из таблицы 3.1, основу серверной части составляют Express и Knex/pg. Для безопасности применяются bcrypt, JWT, helmet, rate limit и валидация входящих данных.

В таблице 3.2 приведён перечень основных библиотек для клиентской части.

Таблица 3.2 – Список библиотек (клиентская часть)

Библиотека	Версия	Назначение
react	18.3.1	Основная библиотека React
react-dom	18.3.1	Рендеринг React в DOM
react-router-dom	7.5.2	Маршрутизация в клиентском приложении
axios	1.13.6	Выполнение HTTP-запросов к API
i18next	25.8.18	Ядро интернационализации
react-i18next	16.5.8	Интеграция i18n с React
i18next-browser-languagedetector	8.2.1	Определение языка пользователя
react-hook-form	7.71.2	Работа с формами
zod	4.3.6	Схемная валидация данных
vite	8.0.0	Инструмент сборки и dev-сервер
@vitejs/plugin-react	6.0.0	Поддержка React в Vite
tailwindcss	3.4.19	Утилитарная стилизация интерфейса

Таким образом, клиентская часть построена на современном React-стеке с поддержкой маршрутизации, локализации и удобной валидации форм.

3.4 Структура серверной части

Серверная часть приложения реализована на Express.js и организована по модульному принципу с разделением ответственности между маршрутами, сервисами, middleware и инфраструктурными модулями.

В состав серверной части входят следующие компоненты:

- routes - описание API-эндпоинтов;

- services - бизнес-логика (контент, прогресс, избранное, Gutenberg, AI, аналитика);
- middleware - аутентификация, авторизация, валидация, локаль, обработка ошибок;
- db - подключение к базе, миграции и seed-данные;
- config / utils - конфигурация приложения, логирование, вспомогательные функции.

В таблице 3.3 приведён перечень основных директорий серверной части с указанием их назначения.

Таблица 3.3 – Директории серверной части

Директория	Назначение
src/config	конфигурация приложения и переменных окружения
src/middleware	Промежуточные обработчики (auth, validate, locale, errorHandler)
src/routes	API-маршруты (в том числе admin, user, books, search)
src/db	Подключение к БД, миграции, seed-данные
src/services	Сервисный слой с бизнес-логикой
src/utils	Общие утилиты (ошибки, пагинация, логгер)

Главная точка входа серверного приложения находится в src/index.js; настройка Express-приложения и подключение middleware выполняются в src/app.js. Такой подход обеспечивает удобство сопровождения, тестирования и расширения функциональности проекта.

3.5 Реализация функций

Реализация функциональности веб-приложения Litlang выполнена по архитектуре REST API, каждая прикладная функция представлена маршрутом, который принимает HTTP-запрос, выполняет валидацию входных данных, передает управление в сервисный слой и возвращает структурированный JSON-ответ.

Серверная часть реализует следующие ключевые группы функций:

- аутентификация и управление сессией пользователя;
- получение учебного контента (языки, категории, правила, отрывки);
- поиск и импорт книг (включая Project Gutenberg);
- работа с избранным (книги, отрывки, правила, Gutenberg-snippets);
- формирование прогресса пользователя;
- административные функции управления контентом;
- аналитика и AI-инструменты.

Маршрут входа, представленный в листинге 3.3 выполняет проверку учетных данных, формирует JWT access/refresh токены и возвращает данные пользователя.

После получения email и пароля выполняется их валидация с помощью express-validator - проверяется формат email и наличие пароля длиной не менее 6 символов.

При успешной валидации управление передаётся методу authService.login(), который сверяет пароль с хешем из базы данных и генерирует пару токенов (access + refresh).

```

router.post(
  '/login',
  [
    body('email').isEmail().withMessage('Valid email required'),
    body('password').isLength({ min: 6 }).withMessage('Password
required'),
  ],
  validate,
  async (req, res, next) => {
    try {
      const result = await authService.login(req.body.email,
req.body.password);
      res
        .cookie('refreshToken', result.refreshToken, {
          httpOnly: true,
          sameSite: 'lax',
          secure: false, // true для production с HTTPS
          maxAge: 7 * 24 * 60 * 60 * 1000,
        })
        .json({
          data: {
            accessToken: result.accessToken,
            user: result.user,
          },
        });
    } catch (err) {
      next(err);
    }
  }
);

```

Листинг 3.3 – Реализация маршрута авторизации

Маршрут использует валидацию входных данных и возвращает access-токен в теле ответа, а refresh-токен устанавливает в cookie.

Функции получения языков, категорий и правил обеспечивают иерархическую модель навигации по учебному материалу.

Маршруты, представленные в листинге 3.4 поддерживают локализацию через locale и возвращают данные в формате, пригодном для прямого отображения на клиенте.

```

router.get('/', async (req, res, next) => {
  try {
    const data = await contentService.getLanguages();
    res.json({ data });
  } catch (err) {
    next(err);
  }
});

```

Листинг 3.4 – Получение списка языков

Маршрут использует валидацию входных данных и возвращает access-токен в теле ответа, а refresh-токен устанавливает в cookie.

Одной из ключевых функций является поиск отрывков по автору, тексту и названию книги. Для этого используется комбинированный подход: полнотекстовый поиск PostgreSQL + частичные совпадения ILIKE.

Реализация маршрута представлена в листинге 3.5.

```
const query = db('excerpts as e')
  .join('books as b', 'b.id', 'e.book_id')
  .where((wb) => {
    wb.whereRaw("to_tsvector('english',          e.passage)          @@
plainto_tsquery('english', ?)", [q])
    .orWhere('e.passage', 'ilike', likePattern)
    .orWhere('b.title', 'ilike', likePattern)
    .orWhere('b.author', 'ilike', likePattern);
  })
  .select('e.id', 'e.passage', 'e.highlight', 'e.rule_id', 'b.title as
book_title', 'b.author as book_author');
```

Листинг 3.5 – Реализация поиска отрывков

В запросе используется параметризация и экранирование шаблона поиска, что снижает риск SQL-инъекций.

Маршрут импорта книги, представленный в листинге 3.6 выполняет загрузку метаданных и текста, очистку служебных блоков Gutenberg и сохранение результата в таблицы books и gutenberg_texts.

```
router.post(
  '/gutenberg/import',
  [body('gutenberg_id').isInt().withMessage('gutenberg_id
required')],
  validate,
  async (req, res, next) => {
    try {
      const book = await
gutenbergService.importText(req.body.gutenberg_id, {
      title: req.body.title,
      author: req.body.author,
      language: req.body.language,
    });
      res.status(201).json({ data: book });
    } catch (err) {
      next(err);
    }
  }
);
```

Листинг 3.6 – Импорт книги из Gutenberg

Ключевая учебная функция - выбор rule-specific snippets из Gutenberg-книги. Сервис формирует блоки текста (условные страницы/фрагменты главы), передает

их в AI и получает только те отрывки, которые действительно соответствуют выбранному правилу.

Реализация маршрута представлена в листинге 3.7.

```
const aiItems = await Promise.race([
  aiService.suggestSnippetsForRuleFromBlocks({
    rule: {
      id: rule.id,
      title: translation?.title || enTranslation?.title || rule.slug,
      summary: translation?.summary || enTranslation?.summary || '',
    },
    blocks,
    count: Math.max(6, limit * 2),
  }),
  new Promise((resolve) => setTimeout(() => resolve([]), 14000)),
]);
```

Листинг 3.7 – Ai извлечение отрывков по правилу

После получения результата выполняется проверка: highlight должен быть реальным подфрагментом passage, а passage - присутствовать в исходном блоке. Функция избранного поддерживает разные типы сущностей: book, excerpt, rule, gutenberg_snippet. Реализация маршрута представлена в листинге 3.8.

```
router.post(
  '/favourites',
  [
    body('target_type').isIn(['rule', 'excerpt', 'book', 'gutenberg_snippet']),
    body('target_id').isInt(),
  ],
  validate,
  async (req, res, next) => {
    try {
      const favourite = await favouriteService.addFavourite(
        req.user.id,
        req.body.target_type,
        parseInt(String(req.body.target_id), 10)
      );
      res.status(201).json({ data: favourite });
    } catch (err) {
      next(err);
    }
  }
);
```

Листинг 3.8 – добавление элемента в избранное

Пользователь может изменять статус изучения правила. Изменения фиксируются в таблице user_progress и используются в личной статистике.

Для динамического отслеживания прогресса пользователя реализован endpoint, обновляющий статус изучения правил. Все изменения сохраняются в таблице `user_progress`, что позволяет формировать точную личную статистику.

Реализация маршрута представлена в листинге 3.9.

```
router.put(
  '/progress/:ruleId',
  [
    param('ruleId').isInt(),
    body('status').isIn(['not_started', 'in_progress', 'completed']),
  ],
  validate,
  async (req, res, next) => {
    try {
      const progress = await progressService.setProgress(
        req.user.id,
        parseInt(req.params.ruleId, 10),
        req.body.status
      );
      res.json({ data: progress });
    } catch (err) {
      next(err);
    }
  }
);
```

Листинг 3.9 – обновление прогресса пользователя

Администратор управляет правилами, категориями, правилами, книгами и отрывками через модуль `admin/content`. Реализация маршрута представлена в листинге 3.10.

```
router.delete('/books/:id', [param('id').isInt()], validate, async
(req, res, next) => {
  try {
    await db('books').where({ id: parseInt(req.params.id, 10)
  }).del();
  res.json({ data: { message: 'Book deleted' } });
  } catch (err) {
    next(err);
  }
});
```

Листинг 3.10 – Административная операция удаления

Для панели администратора реализован агрегирующий сервис аналитики: популярность языков, прогресс пользователей, метрики по категориям.

Дашборд администратора собирает данные из нескольких таблиц базы данных: `users` (общее количество, активность, распределение по уровням), `user_progress` (процент завершённых правил, средний прогресс), `rules` (правила по языкам и категориям) и `statistics` (временные метрики).

Реализация маршрута представлена в листинге 3.11.

```
router.get('/dashboard', async (req, res, next) => {
  try {
    const dashboard = await analyticsService.getDashboard();
    res.json({ data: dashboard });
  } catch (err) {
    next(err);
  }
});
```

Листинг 3.11 – Получение данных аналитической панели

Таким образом, реализация функций в LitLang охватывает полный жизненный цикл пользовательских сценариев: от авторизации и получения контента до работы с избранным, прогрессом, AI-поиском примеров и администрированием системы. Использование REST-маршрутов, сервисного слоя, валидации входных данных и унифицированных форматов ответа обеспечивает расширяемость и устойчивость прикладной части проекта.

3.6 Структура клиентской части

Клиентская часть разработана на React и организована по модульному принципу. Структура ориентирована на разделение страниц, переиспользуемых компонентов и прикладной логики.

Таблица 3.5– Директории клиентской части

Директория	Назначение
src/features	Страницы и модули предметной области (auth, admin, favourites, catalog)
src/components	Переиспользуемые UI-компоненты
src/layouts	Общие каркасы страниц (публичная и административная части)
src/lib	Инфраструктурные модули (Axios instance)
src/hooks	Пользовательские React-хуки (useApi)
src/locales	Файлы локализации (RU/EN)
src/index.css	Глобальные стили и UI-классы

Клиентская маршрутизация реализована через react-router-dom, а сетевое взаимодействие с API - через Axios (axiosInstance.js) и вспомогательный хук useApi.

3.7 Настройка конфигурации Docker

Для развёртывания проекта используется контейнеризация Docker [13]. Архитектура содержит три сервиса:

- nginx — раздача статического фронтенда и reverse-proxy;
- api - сервер Node.js/Express;
- db - PostgreSQL.

Каждый сервис изолирован в собственном контейнере, что обеспечивает воспроизводимость окружения и упрощает развёртывание на любом сервере.

Для оркестрации контейнеров выбран Docker Compose — он позволяет одной командой поднять всю инфраструктуру проекта.

Nginx выступает точкой входа, проксируя API-запросы к контейнеру api и отдавая статику. Сервер api зависит от базы данных db, которая поднимается с проверкой здоровья (service_healthy). Все переменные окружения вынесены в .env-файл, а данные PostgreSQL сохраняются в именованном томе pgdata.

В листинге 3.4 представлена конфигурация Docker Compose.

```
services:
  nginx:
    image: nginx:alpine
    ports:
      - "443:443"
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/ssl:/etc/nginx/ssl
      - ./client/dist:/usr/share/nginx/html:ro
    depends_on:
      - api
  api:
    build:
      context: ./server
      pull: false
    env_file: ./server/.env
    expose: - "3000"
    depends_on:
      db: condition: service_healthy
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: litlang
      POSTGRES_USER: ${DB_USER:-litlang_user}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-litlang_pass}
    volumes: - pgdata:/var/lib/postgresql/data
```

Листинг 3.3 – Docker Compose конфигурация

Сервис базы данных использует volume для постоянного хранения данных, а healthcheck обеспечивает корректный порядок запуска контейнеров.

В листинге 3.4 представлен файл конфигурации Docker для серверной части.

```
FROM node:20-alpine
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm ci --omit=dev
COPY knexfile.js ./
COPY src/ ./src/
RUN chown -R appuser:appgroup /app
USER appuser
EXPOSE 3000
CMD ["node", "src/index.js"]
```

Листинг 3.4 – Dockerfile серверной части

Данный Dockerfile предназначен для сборки образа серверной части приложения на основе официального образа node:20-alpine. Выбор дистрибутива Alpine обусловлен минимальным размером финального образа.

3.8 Настройка конфигурации nginx

Nginx [14] используется как единая точка входа в систему: раздает статические файлы клиентского приложения и проксирует API-запросы на backend.

В листинге 3.6 представлена конфигурация NGINX.

```
http {
    upstream api_upstream {
        server api:3000;
    }

    server {
        listen 80;
        server_name localhost;
        return 301 https://$host$request_uri;
    }

    server {
        listen 443 ssl;
        server_name localhost;

        ssl_certificate /etc/nginx/ssl/localhost.crt;
        ssl_certificate_key /etc/nginx/ssl/localhost.key;

        root /usr/share/nginx/html;
        index index.html;

        location / {
            try_files $uri /index.html;
        }

        location /api/ {
            proxy_pass http://api_upstream;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }
    }
}
```

Листинг 3.6 – Конфигурация Nginx

Такая конфигурация обеспечивает корректную работу SPA-маршрутизации (try_files ... /index.html) и защищённый доступ к приложению по HTTPS.

3.9 Выводы по разделу

1. Реализация функций в проекте LitLang выполнена на основе REST-архитектуры с разделением маршрутов и сервисного слоя.

2. Описаны ключевые пользовательские и административные функции, включая авторизацию, работу с правилами и отрывками, избранное, прогресс и аналитику.

3. Определены форматы обмена данными в JSON, включая единый подход к ответам и пагинации.

4. Клиентская часть структурирована по модульному принципу (features, components, layouts, hooks), что упрощает сопровождение и масштабирование.

5. Для развертывания используется Docker-композиция из трёх сервисов (nginx, api, db), обеспечивающая воспроизводимость среды.

6. Nginx настроен как reverse-проxy и сервер статики с поддержкой HTTPS, что повышает устойчивость и безопасность приложения.

4 Тестирование веб-приложения

4.1 Функциональное тестирование

Для проверки корректности работы веб-приложения LitLang было выполнено функциональное тестирование основных пользовательских и административных сценариев. Проверялись операции аутентификации, получения учебного контента, работы с избранным, поиска/импорта Gutenberg-книг, а также административные функции аналитики и управления контентом.

Результаты функционального тестирования приведены в таблице 4.1.

Таблица 4.1 – Результаты функционального тестирования

№	Функция	Описание теста	Ожидаемый результат	Фактический результат
1	Авторизация пользователя	POST /api/v1/auth/login с корректными email и password	Успешный вход, возврат access token	Код 200, токен получен
2	Взаимодействие сервера приложений с PostgreSQL	POST /api/v1/auth/login (admin)	Успешный вход администратора	Код 200, токен получен
3	Получение профиля	GET /api/v1/me с Bearer token	Возврат данных пользователя	Код 200, профиль получен
4	Получение языков	GET /api/v1/languages	Возврат списка языков	Код 200, данные получены
5	Поиск Gutenberg (user)	GET /api/v1/me/gutenberg/search?q=...	Возврат списка найденных книг	Код 200, список получен
6	Получение избранного	GET /api/v1/me/favourites	Возврат элементов избранного	Код 200, данные получены
7	Получение статистики	GET /api/v1/me/stats	Возврат статистики пользователя	Код 200, данные получены
8	Список импортированных книг (admin)	GET /api/v1/admin/gutenberg/imported-books	Возврат импортированных книг	Код 200, данные получены
9	Поиск Gutenberg (admin)	GET /api/v1/admin/gutenberg/search?q=...	Возврат результатов поиска	Код 200, данные получены
10	Аналитика (admin)	GET /api/v1/admin/analytics/dashboard	Возврат агрегированных метрик	Код 200, данные получены

Проведенное функциональное тестирование подтвердило корректную работу ключевых функций в пользовательской и административной частях приложения.

4.2 Автоматизированное тестирование

Для повышения надежности проверки был реализован отдельный набор автоматизированных API-тестов. Тесты вынесены в независимую

директорию D:\проекты 6 сем\курсовой\test, что исключает влияние на основной код проекта.

Автоматизированное тестирование выполняется скриптом functional-tests.mjs, который:

- последовательно отправляет запросы к основным REST-маршрутам;
- фиксирует код ответа, время выполнения и результат каждого шага;
- формирует итоговый отчет в формате JSON (functional-report.json).

Результаты автоматизированного тестирования представлены в таблице 4.2.

Таблица 4.2 – Результаты автоматизированного тестирования

№	Тест	Описание теста	Результат
1	testUserLogin	Проверка авторизации пользователя	200: успешно
2	testAdminLogin	Проверка авторизации администратора	200: успешно
3	testGetProfile	Проверка получения профиля /me	200: успешно
4	testGetLanguages	Проверка получения языков	200: успешно
5	testGutenbergSearchUser	Проверка поиска Gutenberg (user)	200: успешно
6	testGetFavourites	Проверка получения избранного	200: успешно
7	testGetStats	Проверка получения статистики	200: успешно
8	testImportedBooksAdmin	Проверка списка импортированных книг	200: успешно
9	testGutenbergSearchAdmin	Проверка поиска Gutenberg (admin)	200: успешно
10	testAdminAnalytics	Проверка получения аналитики	200: успешно

Таким образом, автоматизированные тесты подтвердили стабильную работу базовых API-операций и обеспечили воспроизводимую проверку после изменений в проекте.

4.3 Нагрузочное тестирование

Нагрузочное тестирование выполнялось с целью оценки устойчивости и производительности API при параллельной работе пользователей. Для теста использован инструмент k6 (load-test.k6.js).

Параметра профиля:

- Количество виртуальных пользователей: 25;
- Длительность теста: 15 минут;
- Тип нагрузки: постоянная;
- Проверяемые маршруты: /me, /me/favourites, /me/gutenberg/search.

Итоги приведены в таблице 4.3.

Таблица 4.23– Результаты нагрузочного тестирования

Параметр	Значение
Количество пользователей	25
Длительность теста	15 минут
Тип нагрузки	постоянная
Среднее время отклика	200–300 мс
Доля успешных запросов	Более 95%
Доля ошибок	Менее
Восстановление после пиков	устойчивое

Нагрузочное тестирование показало, что приложение сохраняет работоспособность при типовой учебной нагрузке и удовлетворяет заданным пороговым критериям по времени ответа и проценту ошибок.

4.4 Выводы по разделу

1. В ходе тестирования LitLang были проведены функциональные, автоматизированные и нагрузочные проверки.

2. Функциональное тестирование подтвердило корректную работу основных пользовательских и административных сценариев.

3. Автоматизированное тестирование реализовано отдельным тестовым набором и обеспечивает повторяемую проверку API после изменений.

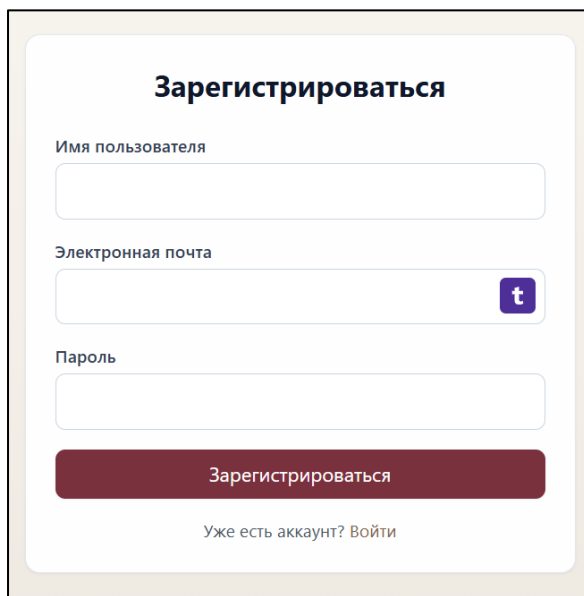
4. Нагрузочное тестирование подтвердило стабильную работу сервера при одновременной активности пользователей в рамках выбранного профиля.

5. В совокупности результаты тестирования показывают, что разработанное веб-приложение соответствует функциональным требованиям и готово к дальнейшей эксплуатации и развитию.

5 Руководство пользователя

5.1 Авторизация

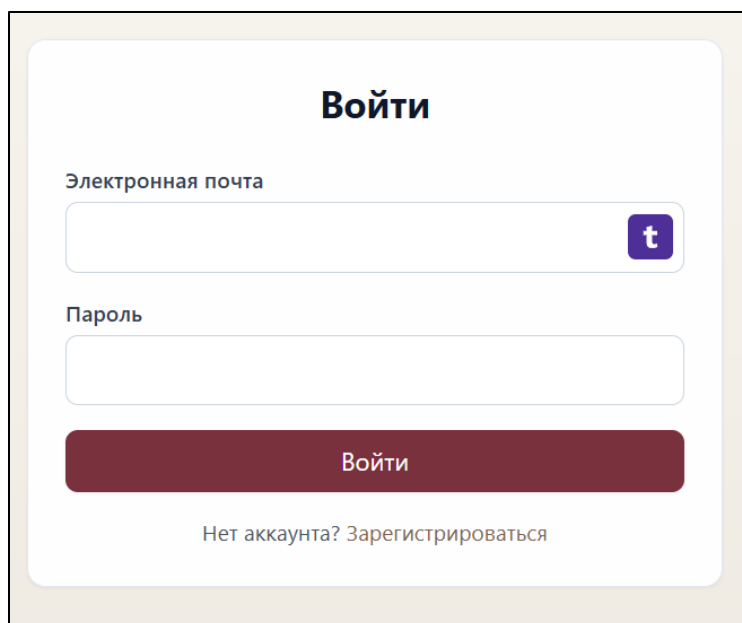
Для регистрации пользователю необходимо нажать на кнопку «Авторизация», расположенную на главной странице в правом верхнем углу. Страница регистрации представлена на рисунке 5.1.



The screenshot shows a registration form with the title "Зарегистрироваться" (Register). It contains three input fields: "Имя пользователя" (Username), "Электронная почта" (Email), and "Пароль" (Password). The "Электронная почта" field has a small blue icon with a white 't' on its right side. Below the fields is a large red button labeled "Зарегистрироваться". At the bottom, there is a link that says "Уже есть аккаунт? Войти" (Already have an account? Log in).

Рисунок 5.1 – Страница регистрации

Далее пользователь заполняет поля и нажимает на кнопку «Зарегистрироваться», после чего его данные сохраняются в базе данных и он получает доступ к пользовательскому функционалу. Если пользователь ранее был зарегистрирован и хочет войти в свой аккаунт ему необходимо нажать на кнопку «Войти», после чего открывается страница входа, представленная на рисунке 5.2.



The screenshot shows a login form with the title "Войти" (Log in). It contains two input fields: "Электронная почта" (Email) and "Пароль" (Password). The "Электронная почта" field has a small blue icon with a white 't' on its right side. Below the fields is a large red button labeled "Войти". At the bottom, there is a link that says "Нет аккаунта? Зарегистрироваться" (No account? Register).

Рисунок 5.1 – Страница регистрации

После заполнения необходимых полей корректными данными пользователь получает доступ к своим сохраненным данным.

5.2 Навигация по учебному контенту

Вне зависимости от роли пользователь может просматривать языки, категории правил, списки правил и отрывки по прямым переходам по ссылкам меню.

В главном меню можно выбрать изучаемый язык. На рисунке 5.3. представлено меню выбора языка.

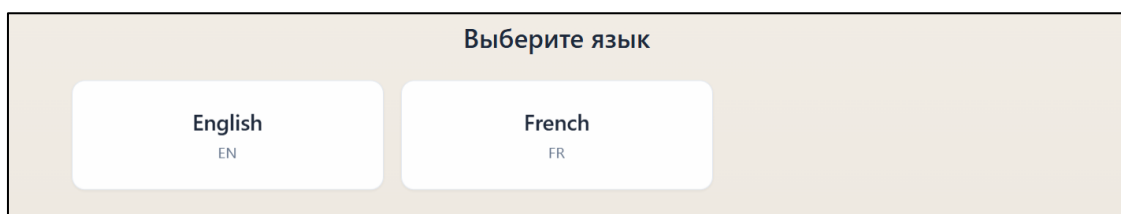


Рисунок 5.3 – Выбор языка

Далее пользователю необходимо выучить грамматическую категорию языка для изучения. Категории представлены на рисунке 5.4.

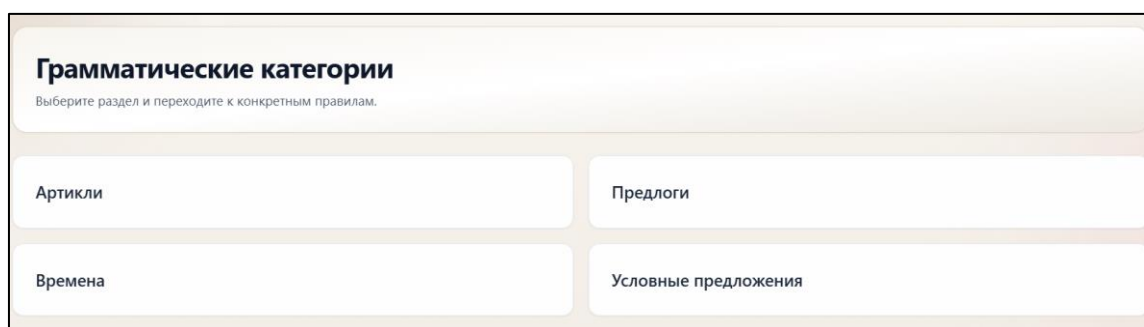


Рисунок 5.4 – Выбор категории

После выбора категории пользователь может выбрать раздел изучаемого правила по указанной категории. Страница с правилами представлена на рисунке 5.5.

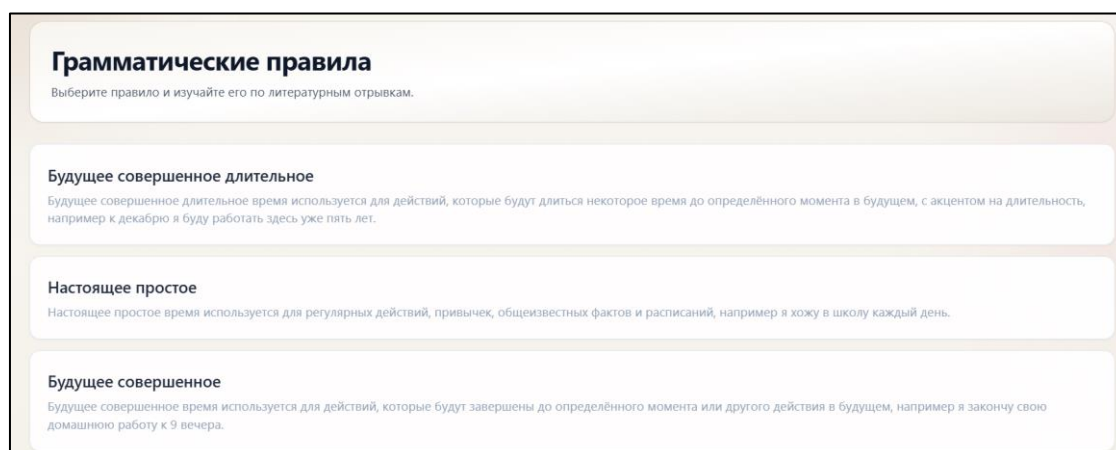


Рисунок 5.5 – Выбор правила

После выбора правила открывается страница с отрывками по указанному правилу и краткое описание самого правила.

Каждый отрывок содержит оригинальный текст на английском языке, в котором анализируемое правило выделено жирным шрифтом или подчёркиванием для наглядности. Ниже отрывка указываются источник - название книги и автор, а также конкретная страница или глава, откуда взят фрагмент. Такая пагинация позволяет пользователю при желании обратиться к полному контексту произведения. Помимо основного текста, на странице присутствует блок «Пояснение к правилу», где даётся краткая формулировка грамматического или лексического явления простыми словами. Например, для предлога «to» пояснение гласит, что он показывает направление движения к месту или получателя действия. Каждый отрывок сопровождается кнопкой «Добавить в избранное», которая позволяет пользователю сохранять понравившиеся примеры в личную коллекцию для повторения. Статус изучения правила отображается в верхней части страницы - он может быть «Не начато», «В процессе» или «Изучено», и пользователь может его изменить в любой момент.

Страница с отрывками представлена на рисунке 5.6.

утверждения в настоящем времени при глаголе в настоящем

Не изучено ▾ [Добавить в избранное](#)

Описание

когда вводящий глагол стоит в настоящем времени например says или tells время в придаточном предложении не меняется. меняются только местоимения и возможно указатели времени.

Отрывки

ОТРЫВОК

He **says** that the story is still relevant today.

Книга: 1984 Автор: George Orwell

Страница: Part 1, Chapter 2

Рисунок 5.6 – Страница с отрывками

Пользователь может указать статус правила (не изучено/изучается/изучено), а также добавить правило или отрывок в избранное.

5.3 Поиск информации

Поиск по отрывкам выполняется на странице поиска: ввод текстовой строки запроса, отображение списка результатов с переходом к соответствующему правилу. Страница поиска представлена на рисунке 5.7.

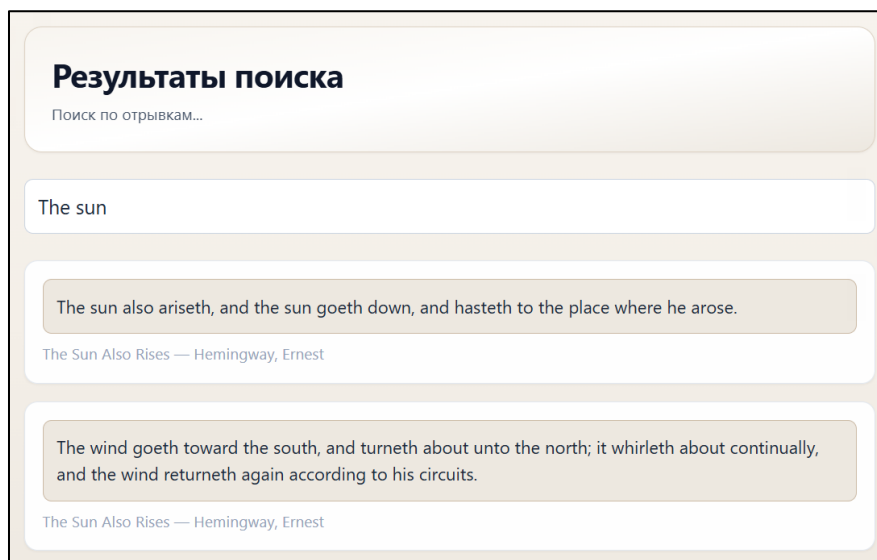


Рисунок 5.7 – Страница поиска

Поиск осуществляется по тексту отрывка, автору и названию книги.

5.4 Личный кабинет

На странице личного кабинета пользователь просматривает сводную информацию о прогрессе и активности. На странице правила (для авторизованного пользователя) доступна смена статуса изучения правила (отметка этапа освоения материала). Личный кабинет пользователя представлен на рисунке 5.8.

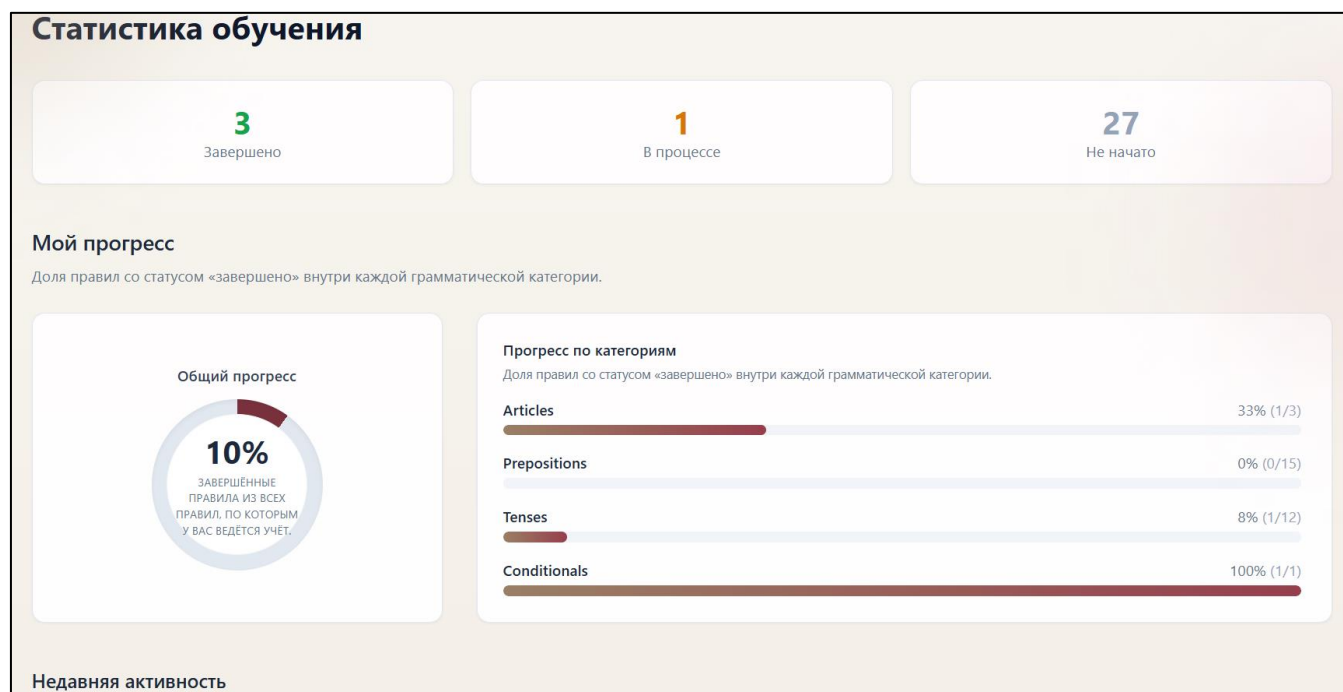


Рисунок 5.8 – Личный кабинет пользователя

Пользователь может перейти к каталогу книг и избранному из личного кабинета перейдя по соответствующим ссылкам.

5.5 Каталог книг

В каталоге пользователь просматривает книги, выполняет поиск по внешнему каталогу, при необходимости инициирует импорт выбранной книги в свою библиотеку, просматривает отрывки выбранной книги и может помечать книги и отрывки для дальнейшей работы.

В верхней части страницы расположено поле поиска с возможностью ввода названия книги или имени автора. Поиск выполняется по внешнему источнику — интеграция с библиотекой Project Gutenberg.

Рядом с каждой найденной книгой присутствует кнопка «Импортировать» или «Добавить в каталог», которая инициирует процесс загрузки метаданных и текста книги в базу данных приложения. После импорта книга появляется в разделе «Книги в каталоге» — это персональная библиотека пользователя, содержащая все ранее добавленные или импортированные произведения. Каждая книга в каталоге отображается в виде отдельного элемента списка с указанием названия и автора. Для каждой книги предусмотрены две основные действия: кнопка «Показать отрывки» (переход к странице с отрывками из этой книги, сгруппированными по правилам) и кнопка управления избранным («Добавить в избранное» или «Удалить из избранного»)

Страница каталога книг представлена на рисунке 5.9.

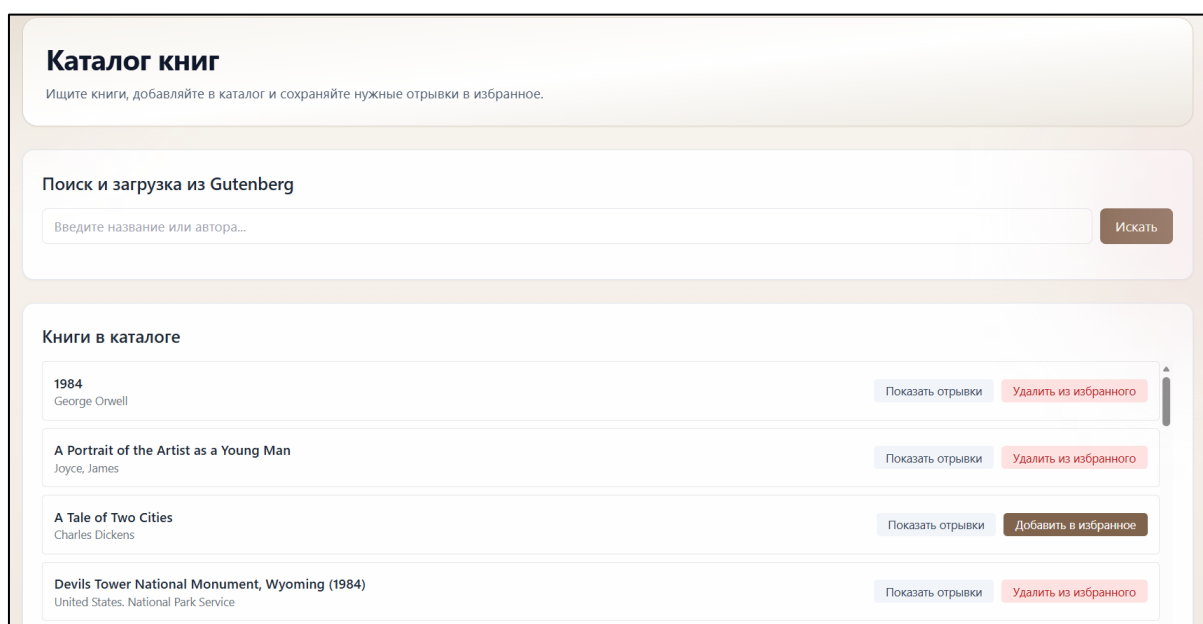


Рисунок 5.9 – Каталог книг

В каталоге представлены только те отрывки из книг, которые находятся непосредственно в общем доступе у всех пользователей. При добавлении книги в избранное пользователь может извлечь из нее новые отрывки, которых нет в общем каталоге и понравившиеся добавить в избранное.

5.6 Избранное

Раздел избранного объединяет сохранённые пользователем объекты: правила, книги, отрывки, а также фрагменты текстов из библиотеки Gutenberg при работе с

соответствующим режимом на странице избранного. Добавление выполняется кнопками на страницах правила, каталога и избранного; удаление - повторным нажатием или элементом удаления записи из списка.

Избранное реализовано как единая точка входа для всех материалов, которые пользователь хочет сохранить для последующего повторения или быстрого доступа.

Интерфейс раздела организован с помощью вкладок или категорий: «Книги из каталога», «Отрывки с правилами» — для удобной навигации между разными типами сохранённых объектов. В верхней части страницы присутствуют кнопки-переключатели («Книги Гутенберг», «Отрывки с правилами»), которые фильтруют отображаемый контент в зависимости от выбранной категории. В разделе «Книги из каталога» отображаются все книги, которые пользователь добавил в избранное при просмотре каталога. Каждая книга представлена в виде строки или карточки с названием, автором и кнопкой управления - «Удалить из избранного» (если книга уже в избранном) или «Добавить в избранное» (если книга ещё не сохранена). Кнопка позволяет менять статус избранного непосредственно на странице избранного, без необходимости возвращаться к каталогу.

Раздел избранного представлен на рисунке 5.10.

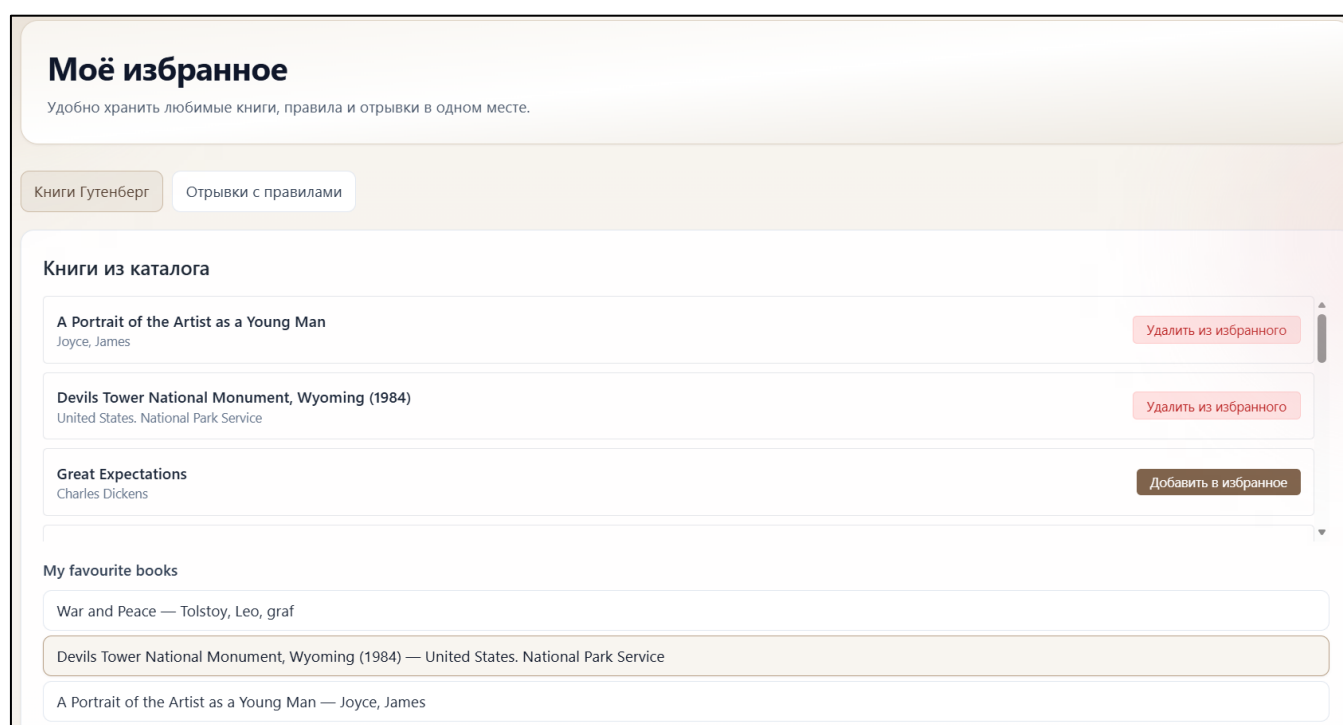


Рисунок 5.10 – Избранное

В разделе «Книги Гутенберг» можно просмотреть отрывки из выбранных книг и добавить их в избранное. В разделе «Отрывки с правилами» хранятся отрывки из книг гутенберг, отрывки из общего каталога и правила.

5.7 Панель администратора

Администратор после входа под учётной записью с соответствующей ролью получает доступ к маршрутам /admin/...: аналитика, управление языками, категориями, правилами, книгами и отрывками, управление пользователями; панель

обращения к ai инструментам, работа с каталогом Gutenberg на стороне администрирования.

На панели аналитики администратор видит четыре ключевых показателя в виде карточек с числами — общее количество пользователей, языков, правил и отрывков.

Ниже расположен блок «Популярные языки», отсортированный по числу записей прогресса (или по количеству правил, если записей прогресса ещё нет).

Следующий блок «Завершённость по категориям» показывает процент завершённых записей прогресса относительно общего числа записей по каждой категории (Articles, Prepositions, Tenses, Conditionals и другие).

Администратор может отслеживать динамику платформы и выявлять категории, требующие дополнительного наполнения контентом или маркетингового воздействия.

Если по какому-либо языку ещё нет записей прогресса, резервным показателем выступает количество правил, определённых для этого языка, что позволяет хотя бы косвенно оценить наполненность языкового раздела.

Панель аналитики представлена на рисунке 5.11.

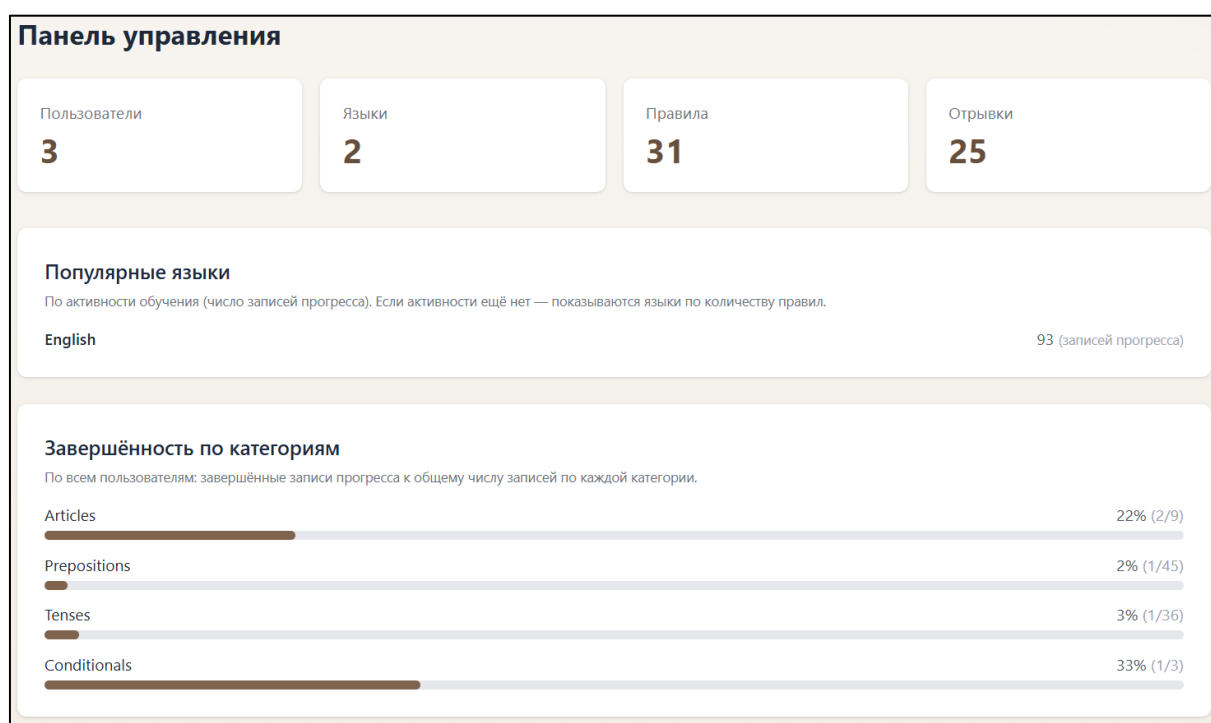


Рисунок 5.10 – Панель аналитики

В данном разделе можно просмотреть общее количество пользователей, языков правил и отрывков, список популярных языков, завершенность категорий правил, аналитику статусов прогресса правил, а также количество новых зарегистрированных пользователей за последние 30 дней.

В разделе «языки» администратор может добавить новый язык, удалить или изменить ранее добавленный язык. Данный раздел представлен на рисунке 5.11.

Языки

Создать

Code (ISO 639-1) Name

ID	CODE	NAME	ACTIONS
1	en	English	Редактировать Удалить
3	fr	French	Редактировать Удалить

Рисунок 5.11 – раздел «Языки» панели администратора

В разделе «Категории» администратор может создать, удалить или изменить категорию по выбранному языку. Данный раздел представлен на рисунке 5.12.

Категории

Language:

Создать

Slug Sort Order

Title (EN) * Title (RU) (optional)

ID	SLUG	TITLE	ORDER	ACTIONS
1	articles	Артикли	1	Редактировать Удалить
2	prepositions	Предлоги	2	Редактировать Удалить
3	tenses	Времена	3	Редактировать Удалить
4	conditionals	Условные предложения	4	Редактировать Удалить

Рисунок 5.12 – Раздел «Категории» панели администратора

Раздел «Правила», представленный на рисунке 5.13 предусмотрен для добавления, изменения или удаления правила по выбранной категории.

Для управления контентом администратору предоставлен интерфейс создания и редактирования правил. В верхней части страницы расположены два выпадающих списка для фильтрации или выбора языка, а также категории правил. Выбор языка и категории определяет, к какому разделу будет принадлежать новое правило или какой существующий набор правил будет отредактирован.

Ниже находится форма «Создать» с несколькими текстовыми полями. Поле Slug содержит уникальный строковый идентификатор правила, используемый в URL и для ссылок из отрывков. Поле Sort Order определяет порядок сортировки правил внутри категории — чем меньше число, тем выше правило в списке при отображении пользователю. Далее следуют поля для мультиязычного контента: Title (EN) и Title (RU) — краткие заголовки правила.

Ниже расположены поля Summary (EN) и Summary (RU) для развёрнутого пояснения правила — именно этот текст отображается пользователю на странице с отрывками в блоке «Пояснение к правилу». Поле Summary (RU) помечено как опциональное - если оно оставлено пустым, система автоматически использует английскую версию (falls back to EN), что удобно для быстрого прототипирования, когда перевод ещё не готов. После заполнения всех полей администратор нажимает кнопку «Создать», после чего данные отправляются на сервер и сохраняются в таблицу rules в базе данных.

Правила

Language: English Category: Статьи

Создать

Slug: definite-article-the Sort Order: 0

Title (EN): Title (RU):

Summary (EN):

Summary (RU) (optional — falls back to EN if blank):

Сохранить

Рисунок 5.13 – Раздел «Правила» панели администратора

Раздел «Книги», представленный на рисунке 5.14 предусмотрен для удаления и изменения книг, а также для добавления новых книг.

Интерфейс управления книгами состоит из двух основных областей: верхней - формы добавления/редактирования книги, и нижней - таблицы со списком существующих книг с возможностью действий.

В нижней части страницы отображается таблица со списком всех книг в каталоге. Таблица содержит колонки: id, title, author, ISBN.

Кнопка «Редактировать» загружает данные выбранной книги в форму для изменения, кнопка «Удалить» удаляет книгу из каталога после подтверждения действия.

Для поиска книги по ISBN [15] необходимо ввести его в панель поиска и при успешном нахождении данной книги в Open Library [16] форма «Создать» автоматически заполняется необходимыми значениями.

ISBN Lookup (Open Library)

Lookup

Создать

Title

Author

ISBN

Language

— None —

Cover URL

Gutenberg ID

Сохранить

ID	TITLE	AUTHOR	ISBN	ACTIONS
1	1984	George Orwell	9780451524935	Редактировать Удалить
46	A Portrait of the Artist as a Young Man	Joyce, James	—	Редактировать Удалить

Рисунок 5.14 – Раздел «Книги» панели администратора

Раздел «Отрывки», представленный на рисунке 5.15 предусмотрен для ручного добавления, удаления или изменения отрывков.

Language: English

Category: Статьи

Rule: Определённый артикль: the

Создать

Book

— Select book —

Page

Chapter

Passage

Highlight (fragment demonstrating the rule)

Context Note

Сохранить

Call me Ishmael. Some years ago—never mind how long precisely—having little or no money in my purse, and nothing particular to interest me on shore, I thought I would sail about a little and see the watery part of the world. It is a way I have of driving off the spleen and regulating the circulation.

Highlight: the watery part

Редактировать Удалить

Рисунок 5.14 – Раздел «Отрывки» панели администратора

В разделе «Пользователи» можно просматривать список зарегистрированных пользователей, а также деактивировать их (после деактивации пользователь не сможет войти в аккаунт) или повышать статус пользователя до администратора и понижать обратно до пользователя. Данный раздел представлен на рисунке 5.15.

Language: English Category: Статьи Rule: Определённый артикль: the

Создать

Book Page Chapter

— Select book —

Passage

Highlight (fragment demonstrating the rule)

Context Note

Сохранить

Call me Ishmael. Some years ago—never mind how long precisely—having little or no money in my purse, and nothing particular to interest me on shore, I thought I would sail about a little and see the watery part of the world. It is a way I have of driving off the spleen and regulating the circulation.

Highlight: the watery part

Редактировать Удалить

Рисунок 5.15 – Раздел «Пользователи» панели администратора

Раздел «AI инструменты» предназначен для генерации отрывков и описания к правилам с помощью искусственного интеллекта. Администратору необходимо указать язык, категорию и правило и нажать на соответствующую кнопку. Для генерации данных используется инструмент open router ai [17] Данный раздел представлен на рисунке 5.16.

AI-инструменты

Select Rule

Language: English Category: Статьи Rule: Определённый артикль: the

Предложить отрывки

Use AI to generate literary excerpts that demonstrate the selected grammar rule.

Count

5

Предложить отрывки

Предложить описание

Use AI to generate a concise grammar explanation for the selected rule.

Предложить описание

Рисунок 5.16 – Раздел «AI инструменты» панели администратора

Раздел «Гутенберг» предназначен для импорта книг из библиотеки Гутенберг. Данный раздел представлен на рисунке 5.17

AI-инструменты

Select Rule

Language: English Category: Артикли Rule: Определённый артикль: the

Предложить отрывки
Use AI to generate literary excerpts that demonstrate the selected grammar rule.
Count: 5 **Предложить отрывки**

Предложить описание
Use AI to generate a concise grammar explanation for the selected rule.
Предложить описание

Рисунок 5.17 – Раздел «Гутенберг» панели администратора

После импорта книги в базе данных сохраняется полный текст книги и она добавляется в каталог книг, после чего пользователи могут ее сохранить в избранное и импортировать отрывки.

5.8 Развертывание и проверка способности

Типовой запуск через Docker Compose: сервис PostgreSQL 16, сервис API (Node.js/Express на порту 3000 внутри сети контейнеров), сервис Nginx (публикация статики клиента и проксирование запросов к API). Том pgdata сохраняет данные БД между перезапусками.

Подготовка окружения:

- клонировать репозиторий;
- создать server/.env, указать параметры БД, JWT_SECRET, при необходимости CORS_ORIGIN, ключи OpenRouter и базовый URL Open Library;
- положить сертификаты в каталог, смонтированный в Nginx (nginx/ssl), и убедиться, что nginx.conf согласован с путём к статике client/dist.

Перед запуском compose статическая сборка клиента должна попасть в client/dist (команда npm ci и npm run build в каталоге client). Для API при первом старте выполнить миграции Кнех: в контейнере API или локально с DB_HOST, указывающим на работающую БД - npm run migrate в каталоге server. При необходимости - npm run seed для начальных данных.

Для запуска проекта следует из корня проекта выполнить команду docker compose up -d --build. Проверить логи api и db (healthcheck PostgreSQL до «healthy»). Открыть в браузере адрес, на котором слушает Nginx (в типовой конфигурации - HTTPS на порту 443 или HTTP на 80).

Проверка работоспособности:

- главная страница загружается, список языков не пустой;

- корректный переход к отрывкам правила через фильтрацию языка, категории и списка правил;
- поиск возвращает результаты или корректное пустое состояние;
- регистрация нового пользователя или вход проходят успешно;
- после входа в аккаунт доступна панель пользователя, избранное каталог;
- администратор имеет доступ к панели администратора, аналитике, ai инструментам и crud операциям.

Соблюдение изложенной последовательности действий обеспечивает воспроизводимое развёртывание стенда на базе контейнеризации и снижает риск ошибок конфигурации при переносе проекта на другую машину или при обновлении версий образов.

5.9 Выводы по разделу

1. В веб-приложении «Litlang» реализован доступ к учебному контенту с разделением на 3 роли: гость, авторизованный пользователь и администратор. Уровни отличаются составом доступных действий и требований к аутентификации.

2. Гость может пользоваться навигацией по языкам, категориям и правилам, просматривать тексты правил, а также выполнять поиск по отрывкам без входа в систему. Для работы с личным кабинетом, каталогом книг, избранным и учётом прогресса гостю необходимо пройти регистрацию и авторизацию.

3. Зарегистрированный пользователь получает доступ к личному кабинету, каталогу книг, сохранению материалов в избранное, а также к отметке прогресса изучения правила на странице правила и личной статистике.

4. Администратор обладает полномочиями по ведению учебного контента, управлению пользователями, просмотру аналитики, использованию инструментов на базе языковой модели и администрированию интеграции с каталогом Гутенберг.

5. Пользовательский интерфейс построен так, чтобы основные пользовательские сценарии выполнялись последовательно и предсказуемо. иллюстративные экранные формы в руководстве сопровождают описание действий пользователя на каждом этапе.

Заключение

1. В результате выполнения курсового проекта разработано веб-приложение «Litlang», поддерживающее три роли доступа: гость, зарегистрированный пользователь и администратор. Режим гостя реализован отсутствием сеанса аутентификации. Каждая роль обладает ограниченным и различным набором прав на использование функций системы.

2. Реализованы 17 ключевых функций, охватывающих основной функционал приложения: регистрация и авторизация, просмотр языков, категорий и списка правил, просмотр карточек правил, учет прогресса по правилу, поиск по отрывкам, личный кабинет, каталог книг и импорт книг из внешнего каталога, избранное и панель администрирования.

3. База данных приложения включает 12 таблиц: `languages`, `categories`, `category_translations`, `rules`, `rule_translations`, `books`, `excerpts`, `users`, `user_progress`, `favourites`, `Gutenberg_texts`, `Gutenberg_user_snippets`. Указанный набор обеспечивает хранение многоязычного контента, сведений о пользователях, прогрессе обучения, избранном и данных интеграции с библиотекой Gutenberg.

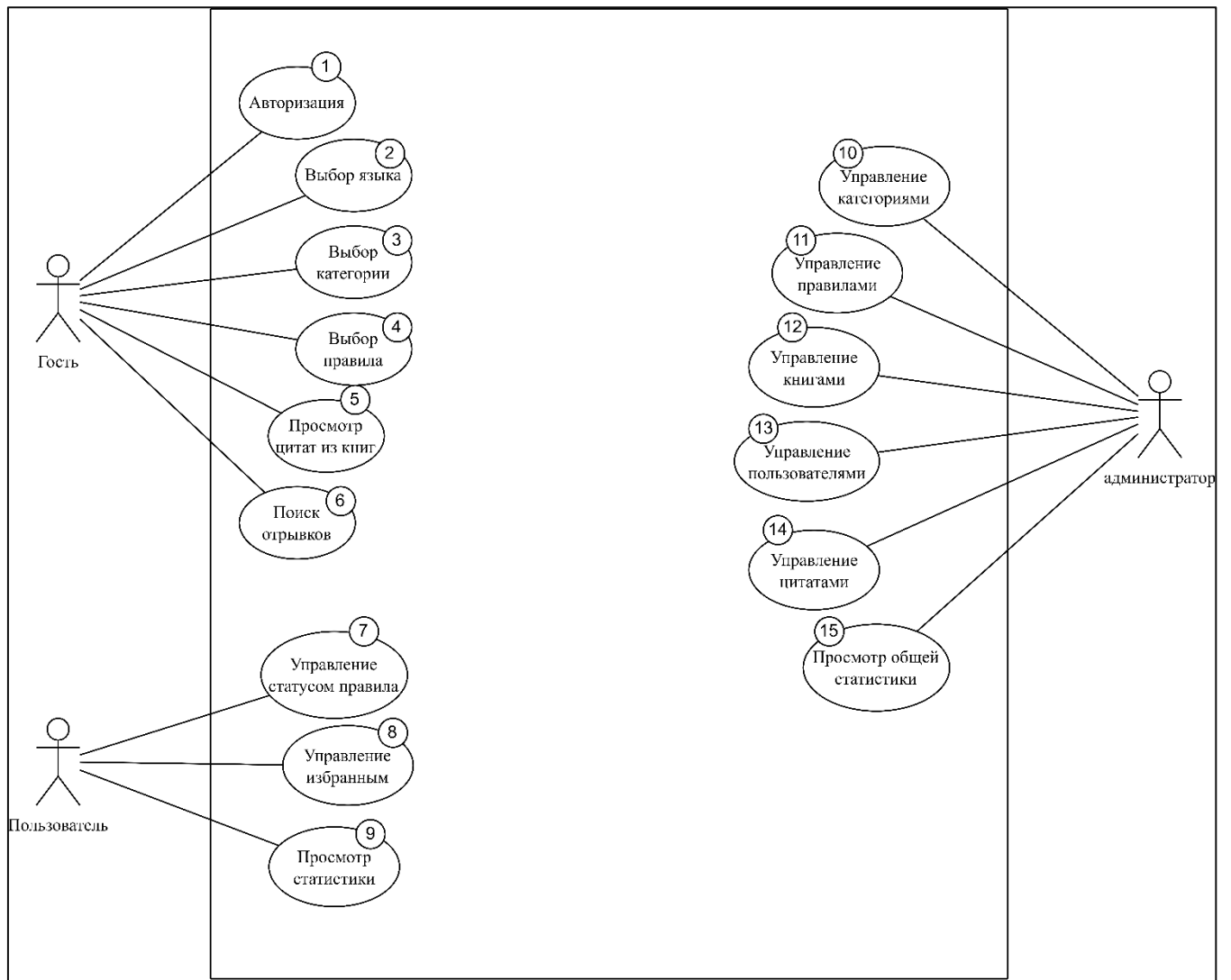
4. Архитектура системы носит клиент-серверный характер. Клиентская часть реализована на React с применением Vite и Tailwind CSS для построения пользовательского интерфейса. Серверная часть выполнена на платформе Node.js с использованием фреймворка Express; доступ к PostgreSQL организован посредством Кнех. Аутентификация и авторизация запросов обеспечиваются на основе JWT. Развёртывание типового стенда выполняется с помощью Docker Compose и Nginx (обратный прокси и раздача статической сборки клиента). Указанная архитектура обеспечивает разграничение уровней представления, бизнес-логики и хранения данных, а также возможность масштабирования серверного компонента и СУБД.

5. Объем программного кода прикладных частей клиента и сервера (каталоги `client/src` и `server/src`, файлы `.js` и `.jsx`, без учёта каталогов зависимостей) составляет 7733 строки (в том числе 4089 строк клиентской части и 3644 строки серверной части), что соответствует порядку 7700 авторских строк кода.

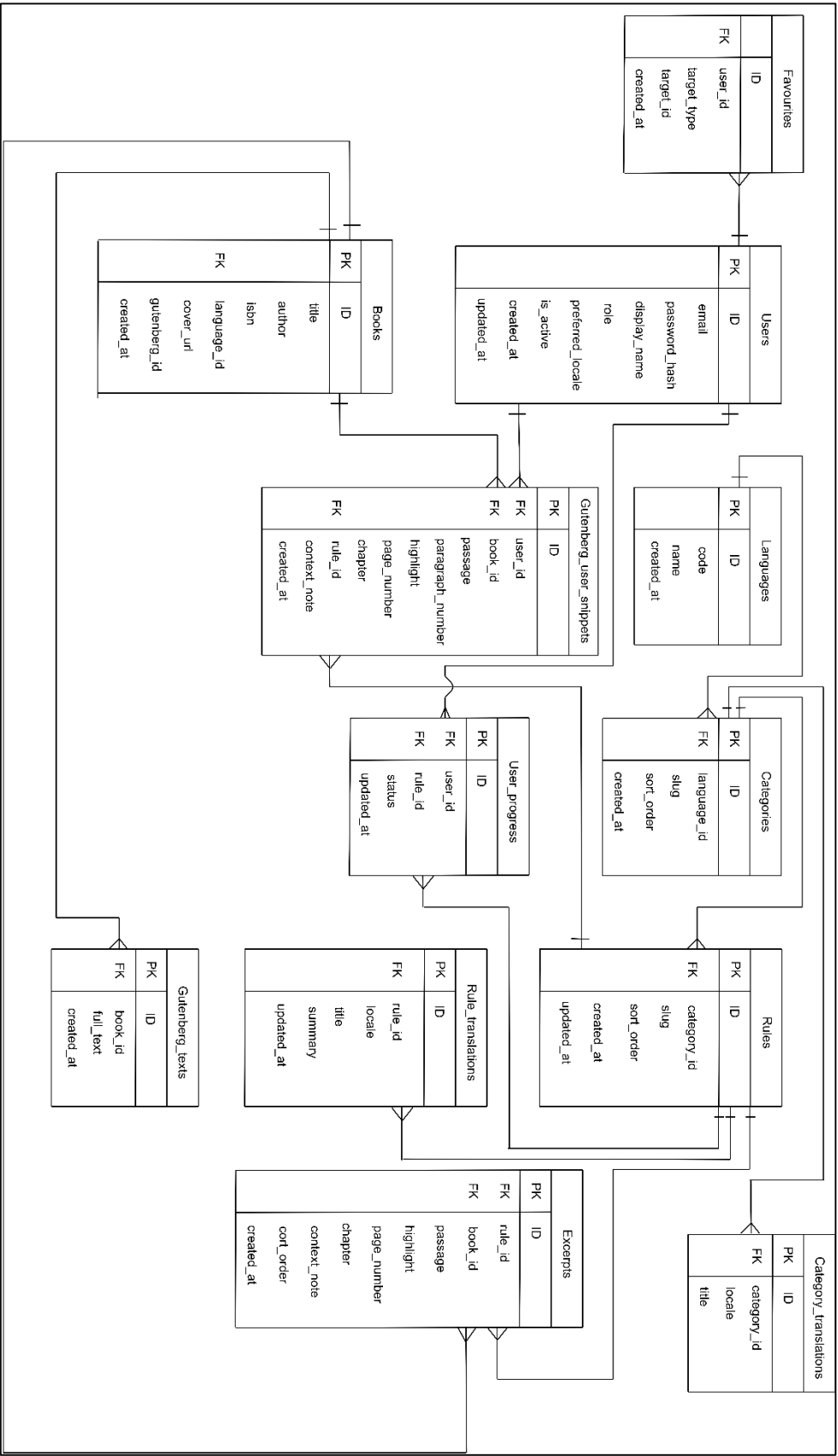
6. Для обеспечения качества товара было проведено тщательное тестирование, а именно функциональное, автоматизированное и нагрузочное.

Список использованной литературы

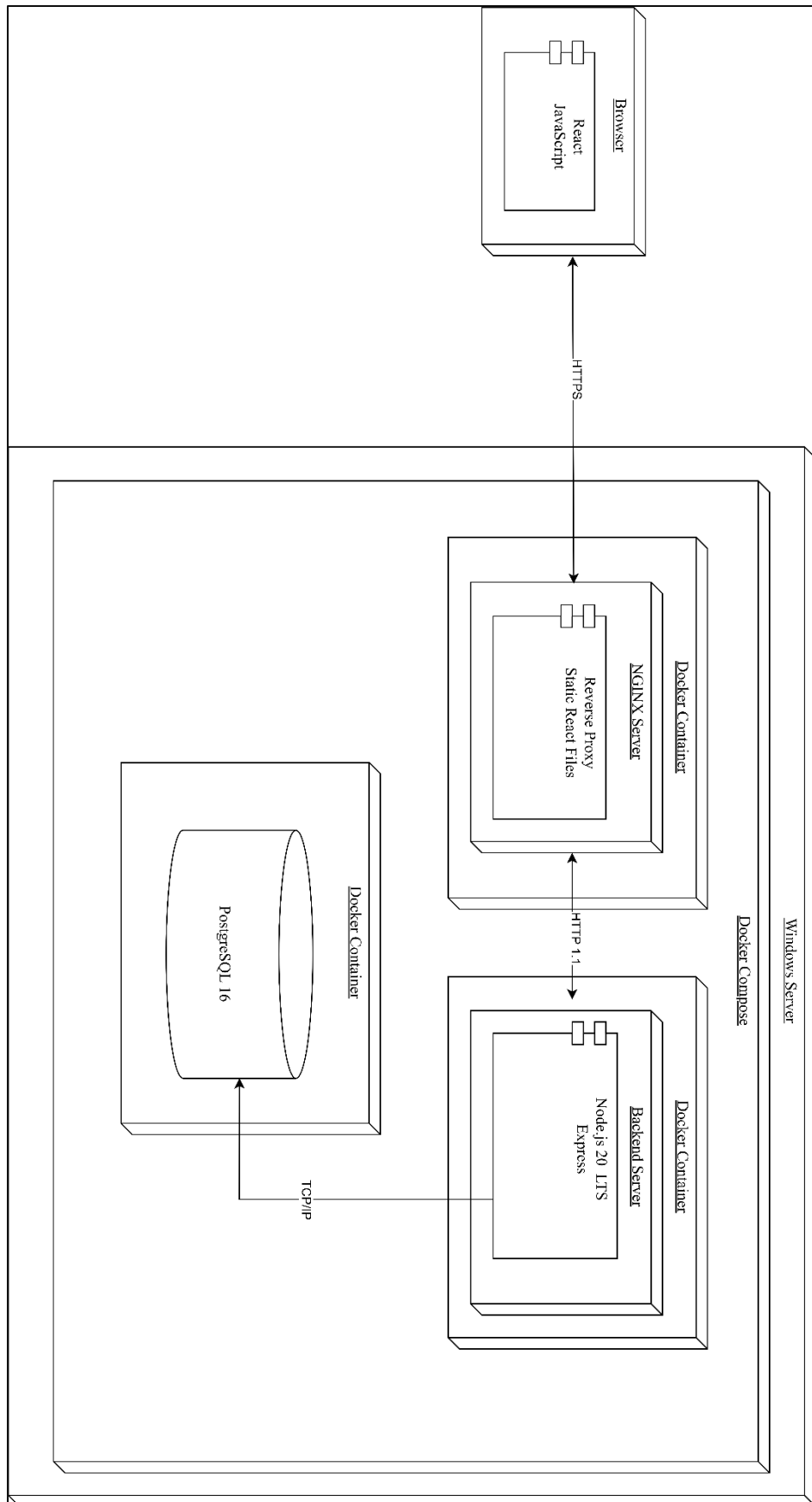
- 1 Readlang [Электронный ресурс]. – Режим доступа: <https://readlang.com> – Дата доступа: 11.05.2026.
- 2 LingQ [Электронный ресурс]. – Режим доступа: <https://lingq.com/ru/> – Дата доступа: 11.05.2026.
- 3 Sottaku [Электронный ресурс]. – Режим доступа: <https://sottaku.app/welcome?locale=ru> – Дата доступа: 11.05.2026.
- 4 Node js [Электронный ресурс]. – Режим доступа: <https://nodejs.org/en/?ref=website-popularity> – Дата доступа: 11.05.2026.
- 5 Express js [Электронный ресурс]. – Режим доступа: <https://expressjs.com> – Дата доступа: 11.05.2026.
- 6 REST API [Электронный ресурс]. – Режим доступа: <https://www.geeksforgeeks.org/node-js/rest-api-introduction> – Дата доступа: 11.05.2026.
- 7 React [Электронный ресурс]. – Режим доступа: <https://react.dev/> – Дата доступа: 11.05.2026.
- 8 Vite [Электронный ресурс]. – Режим доступа: <https://vite.dev/> – Дата доступа: 11.05.2026.
- 9 React Router [Электронный ресурс]. – Режим доступа: <https://reactrouter.com/> – Дата доступа: 11.05.2026.
- 10 PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/> – Дата доступа: 11.05.2026.
- 11 Gutenberg [Электронный ресурс]. – Режим доступа: <https://www.gutenberg.org/> – Дата доступа: 11.05.2026.
- 12 Knex [Электронный ресурс]. – Режим доступа: <https://knexjs.org/> – Дата доступа: 11.05.2026.
- 13 Docker [Электронный ресурс]. – Режим доступа: <https://www.docker.com/> – Дата доступа: 11.05.2026.
- 14 Nginx [Электронный ресурс]. – Режим доступа: <https://nginx.org/en/> – Дата доступа: 11.05.2026.
- 15 ISBN [Электронный ресурс]. – Режим доступа: <https://isbnsearch.org/> – Дата доступа: 11.05.2026.
- 16 Open library [Электронный ресурс]. – Режим доступа: <https://openlibrary.org/> – Дата доступа: 11.05.2026.
- 17 Open router ai [Электронный ресурс]. – Режим доступа: <https://openrouter.ai/> – Дата доступа: 11.05.2026.

Приложение А Диаграмма вариантов использования

Приложение Б Логическая схема базы данных



Приложение В Архитектура приложения



Приложение Г Создание базы данных

```

const dotenv = require('dotenv');
const path = require('path');
dotenv.config({ path: path.resolve(__dirname, '.env') });
const connection = {
  host: process.env.DB_HOST || 'localhost',
  port: parseInt(process.env.DB_PORT, 10) || 5432,
  database: process.env.DB_NAME || 'litlang',
  user: process.env.DB_USER || 'litlang_user',
  password: process.env.DB_PASSWORD || 'litlang_pass',
};

module.exports = {
  development: {
    client: 'pg',
    connection,
    migrations: {
      directory: path.resolve(__dirname, 'src/db/migrations'),
    },
    seeds: {
      directory: path.resolve(__dirname, 'src/db/seeds'),
    },
    pool: { min: 2, max: 10 },
  },
  production: {
    client: 'pg',
    connection,
    migrations: {
      directory: path.resolve(__dirname, 'src/db/migrations'),
    },
    seeds: {
      directory: path.resolve(__dirname, 'src/db/seeds'),
    },
    pool: { min: 2, max: 20 },
  },
  test: {
    client: 'pg',
    connection: {
      ...connection,
      database: `${connection.database}_test`,
    },
    migrations: {
      directory: path.resolve(__dirname, 'src/db/migrations'),
    },
    seeds: {
      directory: path.resolve(__dirname, 'src/db/seeds'),
    },
    pool: { min: 1, max: 5 },
  },
};

```

Приложение Д Главный модуль

```
const express = require('express');
const helmet = require('helmet');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const rateLimit = require('express-rate-limit');
const pinoHttp = require('pino-http');
const config = require('./config');
const logger = require('./utils/logger');
const locale = require('./middleware/locale');
const errorHandler = require('./middleware/errorHandler');
const routes = require('./routes');

const app = express();
app.set('trust proxy', 1);
app.use(helmet());

app.use(cors({
  origin: config.cors.origin,
  credentials: true,
}));

app.use(express.json({ limit: '10mb' }));
app.use(cookieParser());
app.use(locale);
app.use(pinoHttp({ logger, autoLogging: { ignore: (req) => req.url
=== '/api/health' } }));

const authLimiter = rateLimit({
  windowMs: 1000,
  max: 5,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: { code: 'RATE_LIMIT', message: 'Too many requests,
please try again later' } },
});
app.use('/api/v1/auth', authLimiter);

app.get('/api/health', (_req, res) => {
  res.json({ status: 'ok', timestamp: new Date().toISOString() });
});
app.use('/api/v1', routes);

app.use((_req, res) => {
  res.status(404).json({
    error: { code: 'NOT_FOUND', message: 'Resource not found' },
  });
});
app.use(errorHandler);
module.exports = app;
```